

Лабораторна робота №6. Створення вебзастосунка використовуючи фреймворк Flask: розгортання серверу Flask, організація маршрутизації, розроблення представлення з використанням шаблонів, робота з формами, валідація даних, підключення до бази даних та робота с моделями.

Завдання. Ознайомитися з можливостями фреймворку Flask. Виконати встановлення та налаштування. Для заданої предметної галузі та досвіду виконання попередніх лабораторних робіт організувати маршрутизацію запитів, розробити представлення вебзастосунку з використання шаблонізатора Jinja та бібліотеки WTForms. Організувати збереження користувацьких даних з використанням кукі та сесій. Для кожної сутності даних повинні бути реалізовані CRUD операції в залежності від ролі користувача. Реалізувати відношення наступних видів: «один-до-багатьох» - для базового рівня, «один-до-одного» та «багато-до-багатьох» - для розгорнутого рівня. Передбачити можливість застосування міграції даних: у ручному режимі для базового рівня та автоматичному – для розгорнутого.

ЗМІСТ

1 СТВОРЕННЯ БАЗ ДАНИХ У FLASK	3
1.1 SQLAlchemy і Flask-SQLAlchemy	3
1.2 DBAPI	6
1.3 SQLAlchemy і Flask-SQLAlchemy	7
1.4 Створення моделей	8
1.5 Визначення відносин (зв'язків)	10
1.5.1 Відношення один-до-багатьох	11
1.5.2 Відношення один-до-одного	12
1.5.3 Відношення багато-ко-багатьох	13
1.6 Створення таблиць	15
2 ОСНОВИ ORM SQLALCHEMY	19
2.1 Додавання даних	19
2.2 Видалення даних	26
2.3 Запит даних	35
2.3.1 Метод all()	35
2.3.2 Метод count()	37
2.3.3. Метод first()	37
2.3.4 Метод get()	38
2.3.5 Метод get_or_404()	39
2.3.6 Метод filter()	39
2.3.7 Метод first_or_404()	41
2.3.8 Метод limit()	42
2.3.9 Метод offset()	42
2.3.10 Метод order_by()	43
2.3.11 Метод join()	44
2.4 Створення контактної форми	45
3 МІГРАЦІЇ БАЗИ ДАНИХ ЗА ДОПОМОГОЮ ALEMBIC	47
3.1 Розширення Flask-Migrate	48
3.2 Створення скрипта міграції	51

1 СТВОРЕННЯ БАЗ ДАНИХ У FLASK

Суть цієї лабораторної роботи – організація взаємодії вебдодатка з базою даних. Сьогодні існують дві конкуруючі системи баз даних:

- Реляційні бази даних.
- Нереляційні або NoSQL бази даних.

Реляційні бази за традицією використовуються у веб-додатках. Багато великих гравців на ринку веб-програмування все ще використовують їх. Наприклад, Facebook. Реляційні бази даних зберігають дані в таблицях і колонках і використовують зовнішній ключ для створення зв'язку між кількома таблицями. Реляційні бази даних також підтримують транзакції. Це означає, що можна виконати набір SQL-операторів, які мають бути атомарними (atomic). Під atomic маються на увазі всі оператори, які виконуються за принципом "все або нічого".

Останніми роками зросла популярність баз даних NoSQL. Такі бази даних не зберігають дані в таблицях і колонках, а замість них використовують такі структури, як документні сховища, сховища ключів і значень, графи тощо. Більшість NoSQL баз даних не підтримують транзакції, але пропонують більш високу швидкість роботи.

Реляційні бази даних набагато старші за NoSQL. Вони довели свою надійність і безпеку в багатьох галузях. Отже, решту уроку буде присвячено опису принципів використання реляційних баз даних у Flask. Це не означає, що NoSQL не використовуються. Є випадки, коли в NoSQL-базах навіть більше сенсу, але зараз мова піде тільки про реляційні бази даних.

1.1 SQLAlchemy і Flask-SQLAlchemy

SQLAlchemy - це фреймворк для роботи, який на практиці використовується для роботи з реляційними базами даних у Python. Він був створений Майком Байєром у 2005 році. SQLAlchemy підтримує такі бази даних: MySQL, PostgreSQL, Oracle, MS-SQL, SQLite та інші.

SQLAlchemy постачається з потужним ORM (технологія об'єктно-реляційного відображення), який дає змогу працювати з різними базами даних за допомогою об'єктно-орієнтованого коду, а не сирого SQL (мови структурованих запитів). Звичайно, це не зобов'язує використовувати тільки ORM. У будь-який момент можна задіяти можливості SQL.

Flask-SQLAlchemy - це розширення, яке інтегрує SQLAlchemy у фреймворк Flask. Він також пропонує додаткові методи, завдяки яким працювати з SQLAlchemy стає трохи простіше. Встановити Flask-SQLAlchemy разом із додатковими модулями можна за допомогою наступної команди:

```
pip install flask-sqlalchemy
```

```
(c) Корпорація Майкрософт. Усі права захищені.

C:\Users\barba>pip install flask-sqlalchemy
Collecting flask-sqlalchemy
  Downloading flask_sqlalchemy-3.1.1-py3-none-any.whl.metadata (3.4 kB)
Requirement already satisfied: flask>=2.2.5 in c:\python312\lib\site-packages (from flask-sqlalchemy) (3.0.2)
Requirement already satisfied: sqlalchemy>=2.0.16 in c:\python312\lib\site-packages (from flask-sqlalchemy) (2.0.25)
Requirement already satisfied: Werkzeug>=3.0.0 in c:\python312\lib\site-packages (from flask>=2.2.5->flask-sqlalchemy) (3.0.1)
Requirement already satisfied: Jinja2>=3.1.2 in c:\python312\lib\site-packages (from flask>=2.2.5->flask-sqlalchemy) (3.1.3)
Requirement already satisfied: itsdangerous>=2.1.2 in c:\python312\lib\site-packages (from flask>=2.2.5->flask-sqlalchemy) (2.1.2)
Requirement already satisfied: click>=8.1.3 in c:\python312\lib\site-packages (from flask>=2.2.5->flask-sqlalchemy) (8.1.7)
Requirement already satisfied: blinker>=1.6.2 in c:\python312\lib\site-packages (from flask>=2.2.5->flask-sqlalchemy) (1.7.0)
Requirement already satisfied: typing-extensions>=4.6.0 in c:\python312\lib\site-packages (from sqlalchemy>=2.0.16->flask-sqlalchemy) (4.9.0)
Requirement already satisfied: greenlet!=0.4.17 in c:\python312\lib\site-packages (from sqlalchemy>=2.0.16->flask-sqlalchemy) (3.0.3)
Requirement already satisfied: colorama in c:\python312\lib\site-packages (from click>=8.1.3->flask>=2.2.5->flask-sqlalchemy) (0.4.6)
Requirement already satisfied: MarkupSafe>=2.0 in c:\python312\lib\site-packages (from Jinja2>=3.1.2->flask>=2.2.5->flask-sqlalchemy) (2.1.5)
Downloading flask_sqlalchemy-3.1.1-py3-none-any.whl (25 kB)
Installing collected packages: flask-sqlalchemy
Successfully installed flask-sqlalchemy-3.1.1

C:\Users\barba>
```

Або у віртуальне середовище наступними командами:

```
cd c:\flask
python -m venv .venv
.venv\scripts\activate.bat
c:\Flask\.venv\scripts\pip install flask-sqlalchemy
```

```

Administrator: Командная строка
C:\Users\barba>cd c:\Flask
c:\Flask>python -m venv .venv
c:\Flask>.venv\scripts\activate.bat

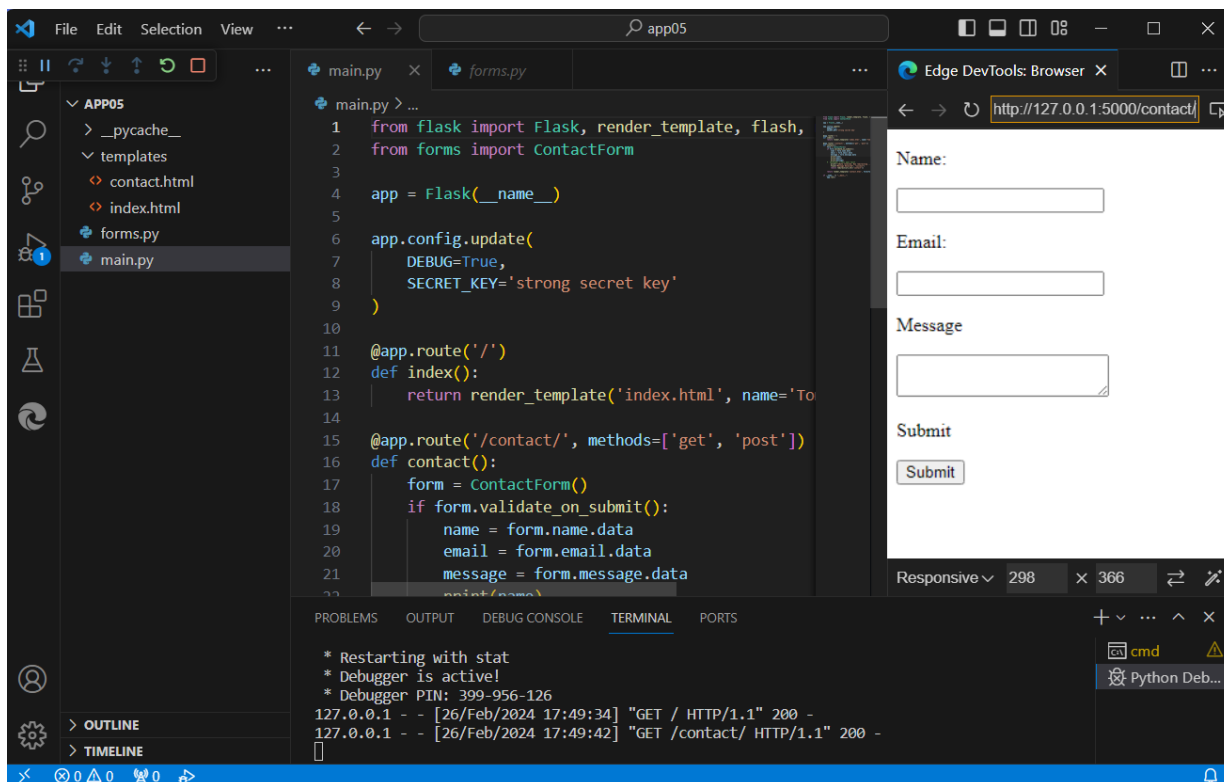
(.venv) c:\Flask>c:\Flask\.venv\scripts\pip install flask-sqlalchemy
Collecting flask-sqlalchemy
  Obtaining dependency information for flask-sqlalchemy from https://files.pythonhosted.org/packages/1d/6a/89963a5c6ecf166e8be29e0d1bf6806051ee8fe6c82e232842e3aeac9204/Flask_sqlalchemy-3.1.1-py3-none-any.whl.metadata
  Downloading flask_sqlalchemy-3.1.1-py3-none-any.whl.metadata (3.4 kB)
Requirement already satisfied: flask<=2.2.5 in c:\Flask\.venv\lib\site-packages (from flask-sqlalchemy) (3.0.2)
Collecting sqlalchemy<=2.0.16 (from flask-sqlalchemy)
  Obtaining dependency information for sqlalchemy<=2.0.16 from https://files.pythonhosted.org/packages/00/2d/7d8f369f22bf08344a266518de9aaa82bce3838245df4c50f436568a0f9/SQLAlchemy-2.0.27-cp312-cp312-win_amd64.whl.metadata
  Downloading SQLAlchemy-2.0.27-cp312-cp312-win_amd64.whl.metadata (9.8 kB)
Requirement already satisfied: Werkzeug<=3.0.0 in c:\Flask\.venv\lib\site-packages (from flask<=2.2.5->flask-sqlalchemy) (3.0.1)
Requirement already satisfied: Jinja2<=3.1.2 in c:\Flask\.venv\lib\site-packages (from flask<=2.2.5->flask-sqlalchemy) (3.1.3)
Requirement already satisfied: itsdangerous<=2.1.2 in c:\Flask\.venv\lib\site-packages (from flask<=2.2.5->flask-sqlalchemy) (2.1.2)
Requirement already satisfied: click<=8.1.3 in c:\Flask\.venv\lib\site-packages (from flask<=2.2.5->flask-sqlalchemy) (8.1.7)
Requirement already satisfied: blinker<=1.6.2 in c:\Flask\.venv\lib\site-packages (from flask<=2.2.5->flask-sqlalchemy) (1.7.0)
Collecting typing-extensions<=4.6.0 (from sqlalchemy<=2.0.16->flask-sqlalchemy)
  Obtaining dependency information for typing-extensions<=4.6.0 from https://files.pythonhosted.org/packages/f9/de/dc04a3ea60b2624b51c703a84bbe0184abcd1d0b9bc8074b5d6b7ab90bb/typing_extensions-4.10.0-py3-none-any.whl.metadata
  Downloading typing_extensions-4.10.0-py3-none-any.whl.metadata (3.0 kB)
Collecting greenlet<=0.4.17 (from sqlalchemy<=2.0.16->flask-sqlalchemy)
  Obtaining dependency information for greenlet<=0.4.17 from https://files.pythonhosted.org/packages/53/80/3d94d5999b4179d91bcc93745d1b0815b073d61be79dd546b840d17adb18/greenlet-3.0.3-cp312-cp312-win_amd64.whl.metadata
  Downloading greenlet-3.0.3-cp312-cp312-win_amd64.whl.metadata (3.9 kB)
Requirement already satisfied: colorama in c:\Flask\.venv\lib\site-packages (from click<=8.1.3->flask<=2.2.5->flask-sqlalchemy) (0.4.6)
Requirement already satisfied: MarkupSafe<=2.0 in c:\Flask\.venv\lib\site-packages (from Jinja2<=3.1.2->flask<=2.2.5->flask-sqlalchemy) (2.1.5)
Downloading flask_sqlalchemy-3.1.1-py3-none-any.whl (25 kB)
Downloading SQLAlchemy-2.0.27-cp312-cp312-win_amd64.whl (2.1 MB)
2.1/2.1 MB 8.8 MB/s eta 0:00:00
Downloading greenlet-3.0.3-cp312-cp312-win_amd64.whl (293 kB)
293.6/293.6 kB 9.1 MB/s eta 0:00:00
Downloading typing_extensions-4.10.0-py3-none-any.whl (33 kB)
Installing collected packages: typing-extensions, greenlet, sqlalchemy, flask-sqlalchemy
Successfully installed flask-sqlalchemy-3.1.1 greenlet-3.0.3 sqlalchemy-2.0.27 typing-extensions-4.10.0

[notice] A new release of pip is available: 23.2.1 -> 24.0
[notice] To update, run: python.exe -m pip install --upgrade pip

(.venv) c:\Flask>

```

Візьмемо за шаблон наступний додаток:



Для використання Flask-SQLAlchemy потрібно імпортувати клас *SQLAlchemy* з пакета *flask_sqlalchemy* і створити екземпляр об'єкта *SQLAlchemy*, передавши йому екземпляр програми. Відкриємо файл *main.py*, щоб змінити код наступним чином:

```
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)

app.config.update(
    DEBUG=True,
    SECRET_KEY='strong secret key'
)

db = SQLAlchemy(app)
```

Екземпляр *db* об'єкта *SQLAlchemy* надає доступ до функцій *SQLAlchemy*.

Далі потрібно повідомити *SQLAlchemy* місце розташування бази даних у вигляді *URI*. Формат *URI* бази даних такий:

```
dialect+driver://username:password@host:port/database
```

- *dialect* посилається на ім'я бази даних, таке як *mysql*, *mssql*, *postgresql* тощо.
- *driver* посилається на DBAPI, який він використовує, щоб з'єднуватися з базою даних. За замовчуванням *SQLAlchemy* працює тільки з *SQLite* без додаткових драйверів. Щоб працювати з іншими базами даних, потрібно встановити конкретний драйвер для бази даних, сумісний з DBAPI.

1.2 DBAPI

DBAPI - це всього лише стандарт, що визначає API Python для доступу до баз даних від різних виробників.

Наступна таблиця містить деякі бази даних і драйвери для них, сумісні з DBAPI:

База даних	Драйвер DBAPI
MySQL	PyMysql

PostgreSQL	Psycopg 2
MS-SQL	pyodbc
Oracle	cx_Oracle

Username і *password* вказуються тільки за потреби. Якщо вказані, вони будуть використовуватися для авторизації в базі даних.

- *host* - місце розташування сервера бази даних.
- *port* - порт сервера бази даних.
- *database* - ім'я бази даних.

Ось деякі приклади URL баз даних для найпопулярніших типів:

```
# URL бази даних для MySQL з використанням драйвера PyMysql
mysql+pymysql://root:pass@localhost/my_db

# URL бази даних для PostgreSQL з використанням psycopg2
postgresql+psycopg2://root:pass@localhost/my_db

# URL бази даних для MS-SQL з використанням драйвера pyodbc
mssql+pyodbc://root:pass@localhost/my_db

# URL бази даних для Oracle з використанням драйвера cx_Oracle
oracle+cx_oracle://root:pass@localhost/my_db

# URL бази даних для SQLite
sqlite:///c:/absolute/path/to/mysql.db
```

Формат URL бази даних для SQLite дещо відрізняється. Оскільки SQLite - це база даних, що базується на файлі, і вона не вимагає імені користувача та пароля, в URL бази даних вказується тільки шлях до файлу бази.

1.3 SQLAlchemy і Flask-SQLAlchemy

Flask-SQLAlchemy використовує конфігураційний ключ `SQLALCHEMY_DATABASE_URI` для визначення URI бази даних. Відкриємо `main2.py`, щоб додати `SQLALCHEMY_DATABASE_URI`:

```
from flask_sqlalchemy import SQLAlchemy
```

```

app = Flask(__name__)

app.config.update(
    DEBUG=True,
    SECRET_KEY='strong secret key',
    SQLALCHEMY_DATABASE_URI = 'mysql+pymysql://admin:masterkey@localhost/app05_db'
)

db = SQLAlchemy(app)

```

У цій частині курсу буде використовуватися база даних *MySQL*. Тому перш ніж переходити до наступного розділу, потрібно переконатися, що *MySQL* працює на комп'ютері.

1.4 Створення моделей

Модель - це клас у Python, який являє собою таблицю бази даних. Її атрибути зіставляються зі стовпцями таблиці. Клас моделі успадковується з *db.Model* і визначає колонки як екземпляри класу *db.Column*.

Відкриємо *main.py*, щоб додати наступний клас:

```

from datetime import datetime

db = SQLAlchemy(app)

class Post(db.Model):
    __tablename__ = 'posts'
    id = db.Column(db.Integer(), primary_key=True)
    title = db.Column(db.String(255), nullable=False)
    slug = db.Column(db.String(255), nullable=False)
    content = db.Column(db.Text(), nullable=False)
    created_on = db.Column(db.DateTime(), default=datetime.utcnow)
    updated_on = db.Column(db.DateTime(), default=datetime.utcnow,
onupdate=datetime.utcnow)

    def __repr__(self):
        return "<{}: {}>".format(self.id, self.title[:10])

```


Тут створюється модель *Post* з 5 змінними класу. Кожна змінна класу, крім `__tablename__`, - це екземпляр класу `db.Column`. `__tablename__` - це спеціальна змінна класу, що використовується для визначення імені таблиці бази даних. За замовчуванням SQLAlchemy не дотримується угоди про створення імен у множині, тому назва таблиці тут - це назва моделі. Якщо на хочеться спиратися на таку поведінку, слід використовувати змінну `__tablename__`, щоб явно вказати назву таблиці.

Перший аргумент конструктора `db.Column()` - це тип колонки, яка створюється.

SQLAlchemy пропонує велику кількість типів колонок, а якщо їх недостатньо, то можна створити свої. Наступна таблиця описує основні типи колонок у SQLAlchemy та їхні відповідні типи в Python і SQL.

SQLAlchemy	Python	SQL
BigInteger	int	BIGINT
Boolean	bool	BOOLEAN или SMALLINT
Date	datetime.date	DATE
DateTime	datetime.datetime	DATETIME
Integer	int	INTEGER
Float	float	FLOAT или REAL
Numeric	decimal.Decimal	NUMERIC
Text	str	TEXT

Також можна задати додаткові обмеження для колонки, передавши їх у вигляді аргументів-ключових слів конструктору `db.Column`. Наступна таблиця містить деякі широко використовувані обмеження:

Обмеження	Опис
<i>nullable</i>	Коли значення дорівнює <i>False</i> , робить колонку обов'язковою. Значення за замовчуванням - <i>True</i> .
<i>default</i>	Створює значення за замовчуванням для колонки.
<i>index</i>	Логічний атрибут. Якщо <i>True</i> , створює індексовану колонку.
<i>onupdate</i>	Створює значення за замовчуванням для колонки при оновленні запису.
<i>primary_key</i>	Логічний атрибут. Якщо <i>True</i> , позначає колонку основним ключем таблиці.
<i>unique</i>	Логічний атрибут. Якщо <i>True</i> , кожна колонка має бути унікальною.

Наприкінці коду було визначено метод `__repr__()`. Він не необхідний, але якщо є, то створює рядкове представлення об'єкта.

Барбарук Віктор Миколайович. Сучасні технології розробки WEB-застосунків з використанням мови Python

Можна було помітити, що значеннями за замовчуванням для *created_on* і *updated_on* обрано назву методу (*datetime.utcnow*), а не його виклик (*datetime.utcnow()*). Так зроблено, тому що під час виконання коду викликати метод *datetime.utcnow()* немає необхідності. Замість цього його варто викликати, коли запис додається або оновлюється.

1.5 Визначення відносин (зв'язків)

У минулій темі було створено модель *Post* з парою полів. На практиці класи моделей існують самі по собі. Більшу частину часу вони пов'язані з іншими моделями різними типами відносин: один-до-одного, один-до-багатьох, багато-до-багатьох.

Варто далі попрацювати над аналогією блогу. Зазвичай, пост у блозі належить до однієї категорії і має один або кілька тегів. Іншими словами, є відношення один-до-одного між категорією і постом і відношення багато-багато-багато між постом і тегом.

Відкриємо *main.py*, щоб додати моделі *Category* і *Tag*:

```
class Category(db.Model):
    __tablename__ = 'categories'
    id = db.Column(db.Integer(), primary_key=True)
    name = db.Column(db.String(255), nullable=False)
    slug = db.Column(db.String(255), nullable=False)
    created_on = db.Column(db.DateTime(), default=datetime.utcnow)

    def __repr__(self):
        return "<{}:{}]>".format(id, self.name)

class Tag(db.Model):
    __tablename__ = 'tags'
    id = db.Column(db.Integer(), primary_key=True)
    name = db.Column(db.String(255), nullable=False)
    slug = db.Column(db.String(255), nullable=False)
    created_on = db.Column(db.DateTime(), default=datetime.utcnow)

    def __repr__(self):
        return "<{}:{}]>".format(id, self.name)
```

1.5.1 Відношення один-до-багатьох

Для створення відношення один-ко-багатьох потрібно розмістити зовнішній ключ у дочірній таблиці. Це найпоширеніший тип відносин. Для створення відношення один-ко-багатьох в SQLAlchemy потрібно виконати такі кроки:

1. Створити новий екземпляр *db.Column* за допомогою обмеження *db.ForeignKey* у дочірньому класі.
2. Визначити нову властивість за допомогою інструкції *db.relationship* у батьківському класі. Ця властивість буде використовуватися для отримання доступу до пов'язаних об'єктів.

Відкриємо *main.py*, щоб змінити моделі *Post* і *Category*:

```
# ...

class Post(db.Model):
# ...
    updated_on = db.Column(db.DateTime(), default=datetime.utcnow,
onupdate=datetime.utcnow)
    category_id = db.Column(db.Integer(), db.ForeignKey('categories.id'))
# ...

class Category(db.Model):
# ...
    created_on = db.Column(db.DateTime(), default=datetime.utcnow)
    posts = db.relationship('Post', backref='category')
# ...
```

Тут для моделі *Post* у *Category* було додано два нові атрибути: *posts* і *category_id*. *db.ForeignKey()* приймає ім'я стовпця, зовнішній ключ якого використовується. Тут значення *categories.id* передається виключенню *db.ForeignKey()*. Це означає, що атрибут *category_id* у *Post* може набувати значення тільки у колонки *id* таблиці *categories*.

Далі в моделі *Category* є атрибут *posts*, визначений інструкцією *db.relationship()*. *db.relationship()* використовується для додавання двонаправленого зв'язку. Іншими словами, вона додає атрибут класу моделі для доступу до пов'язаних об'єктів. Барбарук Віктор Миколайович. Сучасні технології розробки WEB-застосунків з використанням мови Python

Простими словами, вона приймає щонайменше один позиційний аргумент, який є ім'ям класу на іншій стороні відносин.

```
class Category(db.Model):
# ...
    posts = db.relationship('Post')
# ...
```

Наприклад, якщо є об'єкт *Category* (скажімо, *c*), тоді доступ до всіх постів можна отримати за допомогою *c.posts*. А що, якщо потрібно отримати дані з іншого боку, тобто, отримати категорію в об'єкта поста? Для цього використовується *backref*. Так, код:

```
class Category(db.Model):
# ...
    posts = db.relationship('Post', backref='category')
# ...
```

додає атрибут *category* об'єкту *Post*. Це означає, що якщо є об'єкт *Post* (наприклад, *p*), тоді доступ до категорії можна отримувати за допомогою *p.category*.

Атрибути *category* і *posts* у об'єктів *Post* і *Category* існують тільки для зручності. Вони не є реальними колонками в таблиці.

Варто зазначити, що на відміну від атрибута, представленого зовнішнім ключем (який має бути визначений на стороні "багато" у відносинах), *db.relationship()* можна визначати з будь-якого боку.

1.5.2 Відношення один-до-одного

Створення відношення один-до-одного в SQLAlchemy - це майже те ж саме, що і відношення один-до-багатьох. Єдина відмінність - те, що інструкції *db.relationship()* передається додатковий аргумент *uselist=False*. Наприклад:

```
class Employee(db.Model):
```

```

__tablename__ = 'employees'
id = db.Column(db.Integer(), primary_key=True)
name = db.Column(db.String(255), nullable=False)
designation = db.Column(db.String(255), nullable=False)
doj = db.Column(db.Date(), nullable=False)
dl = db.relationship('DriverLicense', backref='employee', uselist=False)

class DriverLicense(db.Model):
    __tablename__ = 'driverlicense'
    id = db.Column(db.Integer(), primary_key=True)
    license_number = db.Column(db.String(255), nullable=False)
    renewed_on = db.Column(db.Date(), nullable=False)
    expiry_date = db.Column(db.Date(), nullable=False)
    employee_id = db.Column(db.Integer(), db.ForeignKey('employees.id')) # Foreign key

```

Примітка: у цих класах передбачається, що у співробітника (*employee*) не може бути більше одного водійського посвідчення (*driver license*). Тому відносини між співробітником і правами - один-до-одного.

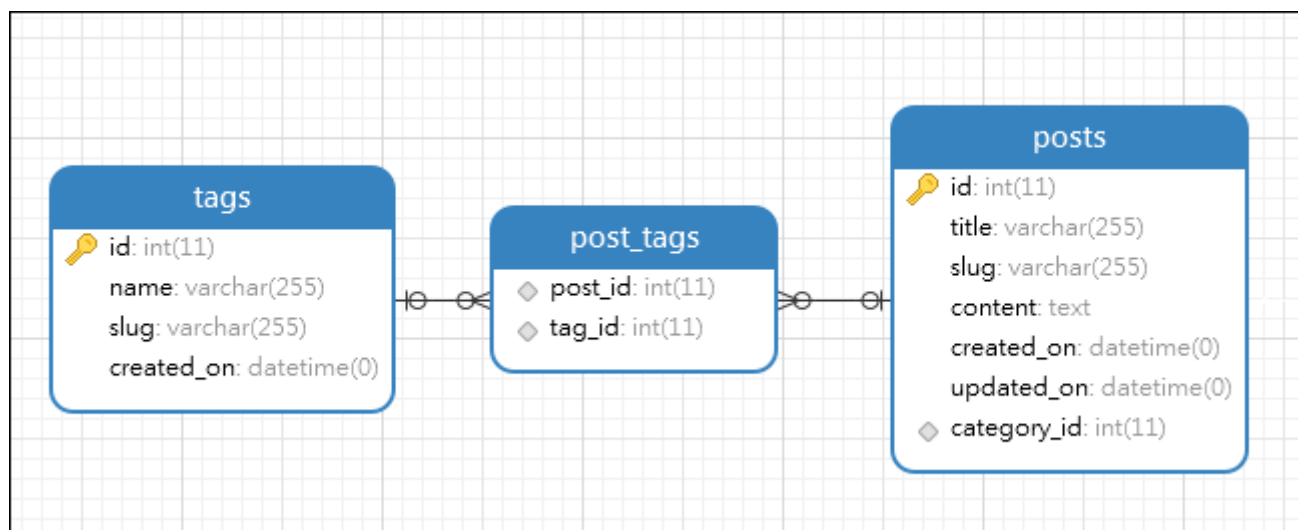
З об'єктом *Employee* можна використовувати *e.dl*, щоб повернути об'єкт *DriverLicense*. Якщо не передати інструкції *db.relationship()* значення *uselist=False*, тоді між *Employee* і *DriverLicense* буде встановлено відношення один-до-багатьох, і *e.dl* поверне список об'єктів *DriverLicense*, замість одного об'єкта. При цьому аргумент *uselist=False* не вплине на атрибут *employee* об'єкта *DriverLicense*. Як і зазвичай, він поверне один об'єкт.

1.5.3 Відношення багато-ко-багатьох

Відношення багато-ко-багатьох вимагає додаткової асоціативної таблиці. Як приклад можна взяти блог.

Пост у блозі зазвичай має один або кілька тегів. Аналогічним чином один тег може асоціюватися з одним або кількома постами. Так утворюється відношення між *posts* і *tags*. Недостатньо додати зовнішній ключ, що посиляється на *id* постів, тому що тег може мати один або кілька постів.

Як рішення потрібно створити нову таблицю асоціацій, визначивши 2 зовнішні ключі, що посиляються на колонки *post.id* і *tag.id*.



Як видно на зображенні, відношення багато-ко-багатьох між постом і тегом створюється за допомогою двох відношень один-до-одного. Перше таке відношення встановлено між таблицями *posts* і *post_tags*, друге - між *tags* і *post_tags*.

Наступний код демонструє, як створити відношення багато-ко-множині в SQLAlchemy. Відкриємо файл *main.py*, щоб додати наступний код:

```

class Category(db.Model):
# ...

    def __repr__(self):
        return "<{}: {}>".format(id, self.name)

post_tags = db.Table('post_tags',
    db.Column('post_id', db.Integer, db.ForeignKey('posts.id')),
    db.Column('tag_id', db.Integer, db.ForeignKey('tags.id'))
)
# ...
class Tag(db.Model):
# ...
    created_on = db.Column(db.DateTime(), default=datetime.utcnow)
    posts = db.relationship('Post', secondary=post_tags, backref='tags')
# ...
  
```

На рядках 7-10 таблиця асоціацій визначається у вигляді об'єкта *db.Table()*. Перший аргумент таблиці *db.Table()* - ім'я таблиці, а додаткові аргументи - це

колонки, представлені екземплярами *db.Column()*. Синтаксис для створення таблиці асоціацій може здатися дивним, якщо порівнювати з процесом створення класу моделі. Це тому що таблиця асоціацій створюється за допомогою SQLAlchemy Core - ще одного елемента SQLAlchemy.

Далі потрібно повідомити клас моделі про таблицю асоціацій, яка буде використовуватися. За це відповідає аргумент-ключове слово *secondary*. На 18 рядку *db.relationship()* викликається з аргументом *secondary*, значення якого - *post_tags*. Хоча відношення було визначено в моделі *Tag*, його можна так само просто визначити в моделі *Post*.

Якщо є, наприклад, об'єкт *p* класу *Post*, тоді доступ до всіх його тегів можна отримати за допомогою *p.tags*. За допомогою об'єкта класу *Tag* (*t*), доступ до постів можна отримати командою *t.posts*.

Настав час створити базу даних і таблиці.

1.6 Створення таблиць

Щоб виконати всі кроки цієї теми, потрібно переконатися, що MySQL встановлено на комп'ютері.

Варто нагадати, що за замовчуванням SQLAlchemy працює тільки з базою даних SQLite. Для роботи з іншими базами даних потрібно встановити драйвер, сумісний з DBAPI. Для використання MySQL підійде драйвер PyMySQL.

```
pip install pymysql
```

```

C:\WINDOWS\system32\cmd.exe  Командная строка
Microsoft Windows [Version 10.0.22621.3155]
(c) Корпорація Майкрософт. Усі права захищені.

C:\Users\barba>cd c:\flask

c:\Flask>python -m venv .venv

c:\Flask>.venv\scripts\activate.bat

(.venv) c:\Flask>cd .venv\scripts\

(.venv) c:\Flask\.venv\Scripts>pip install pymysql
Collecting pymysql
  Obtaining dependency information for pymysql from https://files.pythonhosted.org/packages/e5/30/20/467e39523d0cfc2b6227902d3687a16364307260c75e6a1cb4422b0c62/PyMySQL-1.1.0-py3-none-any.whl.metadata
  Downloading PyMySQL-1.1.0-py3-none-any.whl.metadata (4.4 kB)
  Downloading PyMySQL-1.1.0-py3-none-any.whl (44 kB)
  44.8/44.8 kB 437.1 kB/s eta 0:00:00
Installing collected packages: pymysql
Successfully installed pymysql-1.1.0

[notice] A new release of pip is available: 23.2.1 -> 24.0
[notice] To update, run: python.exe -m pip install --upgrade pip

(.venv) c:\Flask\.venv\Scripts>

```

Після цього необхідно авторизуватися на сервері MySQL (наприклад, за допомогою *MySQL 8.3 Command Line Client - Unicode*) і створити базу даних *app05_db* за допомогою такої команди:

```
CREATE DATABASE flask_app_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

```

MySQL 8.3 Command Line Cli
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 37
Server version: 8.3.0 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE app05_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
Query OK, 1 row affected (0.01 sec)

mysql> |

```


Ця команда створює базу даних *app05_db* з повною підтримкою Unicode.

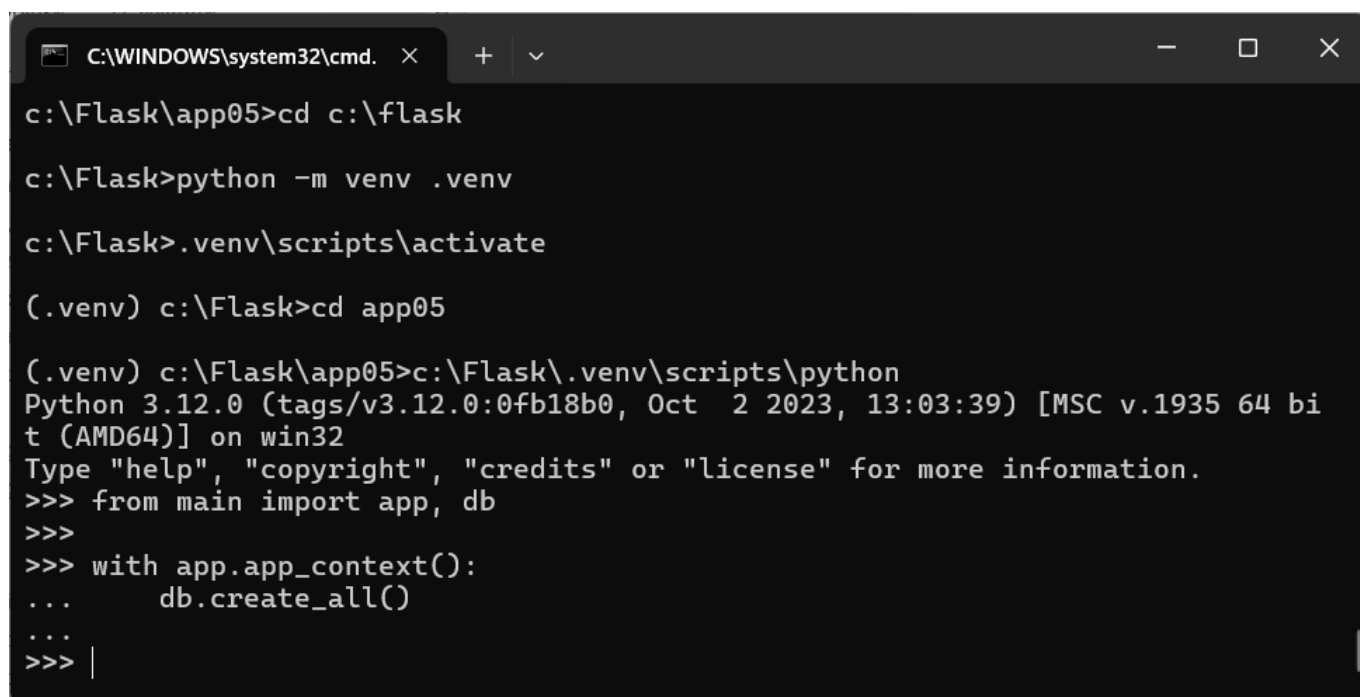
Для створення необхідних таблиці потрібно запустити метод *create_all()* об'єкта SQLAlchemy - *db*. Далі потрібно запустити віртуальне оточення, якщо воно ще не було запущено та активоване, та оболонку Python:

```
cd c:\flask
python -m venv .venv
.venv\scripts\activate.bat

cd app05
c:\Flask\.venv\scripts\python
```

Потім виконати таку команду

```
from main import app, db
with app.app_context():
    db.create_all()
```

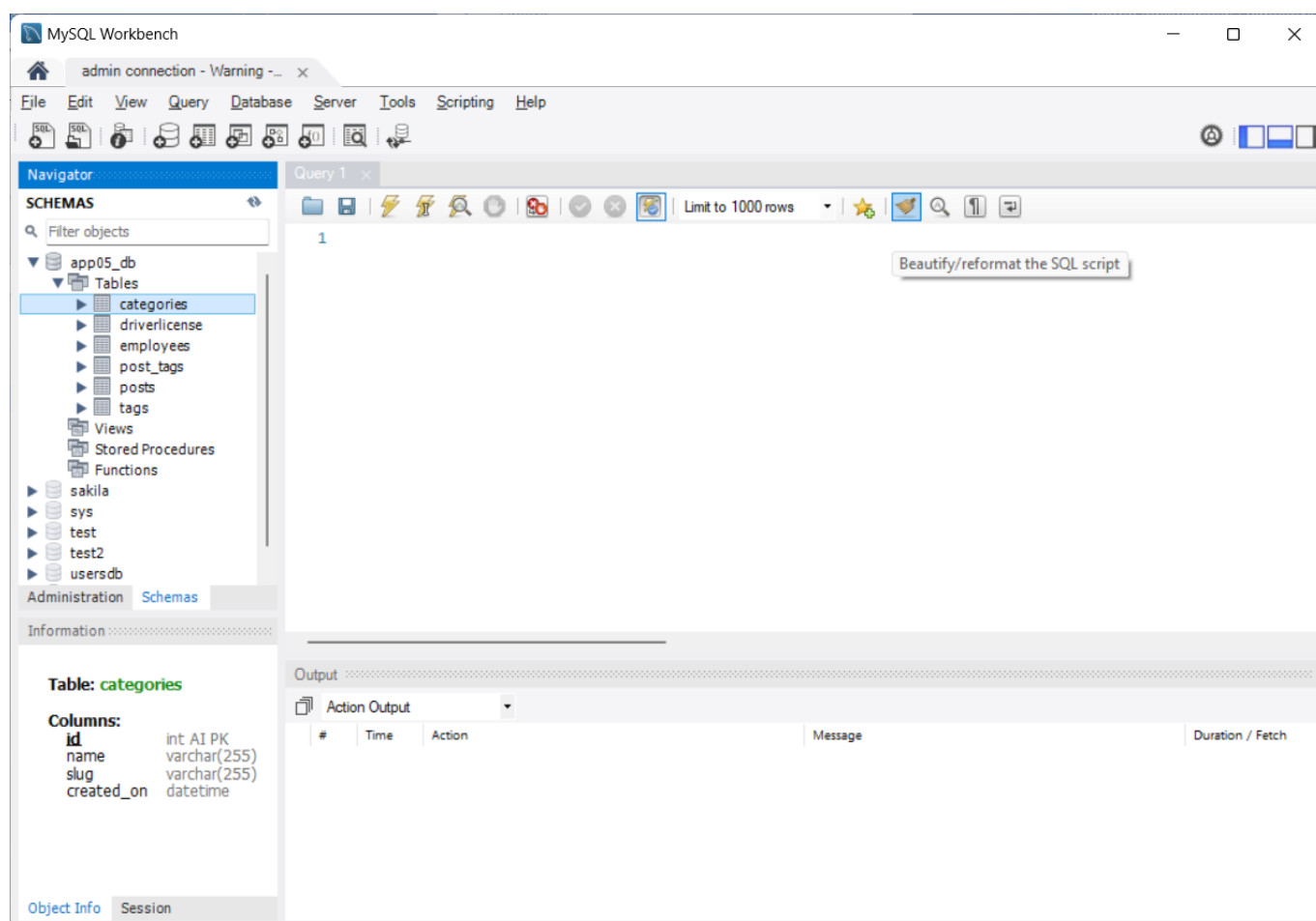


```
C:\WINDOWS\system32\cmd. x + v
c:\Flask\app05>cd c:\flask
c:\Flask>python -m venv .venv
c:\Flask>.venv\scripts\activate
(.venv) c:\Flask>cd app05
(.venv) c:\Flask\app05>c:\Flask\.venv\scripts\python
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct 2 2023, 13:03:39) [MSC v.1935 64 bi
t (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> from main import app, db
>>>
>>> with app.app_context():
...     db.create_all()
...
>>> |
```

Метод *create_all()* створює таблиці тільки в тому випадку, якщо їх немає в базі даних. Тому запускати його можна кілька разів. Також цей метод не бере до уваги зміни моделей під час створення таблиць. Це означає, що якщо запустити метод Барбарук Віктор Миколайович. Сучасні технології розробки WEB-застосувань з використанням мови Python

`create_all()` після зміни його методу, коли таблиця вже створена, то він не змінить схему таблиці. Щоб зробити це, потрібно скористатися інструментом перенесення Alembic. Про те, як переносити бази даних за допомогою Alembic, буде розказано в окремій темі "Перенесення бази даних за допомогою Alembic".

Щоб подивитися створені таблиці, потрібно авторизуватися на сервері MySQL за допомогою MySQL Workbench



База даних *app05_db* має 4 таблиці. Таблиці з назвами *categories*, *posts* і *tags* створені прямо з моделей, а *post_tags* - це таблиця асоціацій, яка являє собою відношення багато-ко-множиною між моделями *Post* і *Tag*.

Клас SQLAlchemy також визначає метод `drop_all()`, який використовується для видалення всіх таблиць у базі даних. Варто пам'ятати, що метод `drop_all()` не враховує, є дані в таблиці чи ні. Він видаляє всі дані, тому використовувати його потрібно з розумом.

Барбарук Віктор Миколайович. Сучасні технології розробки WEB-застосунків з використанням мови Python

2 ОСНОВИ ORM SQLALCHEMY

Усі таблиці на місці. Час додати в них якісь дані.

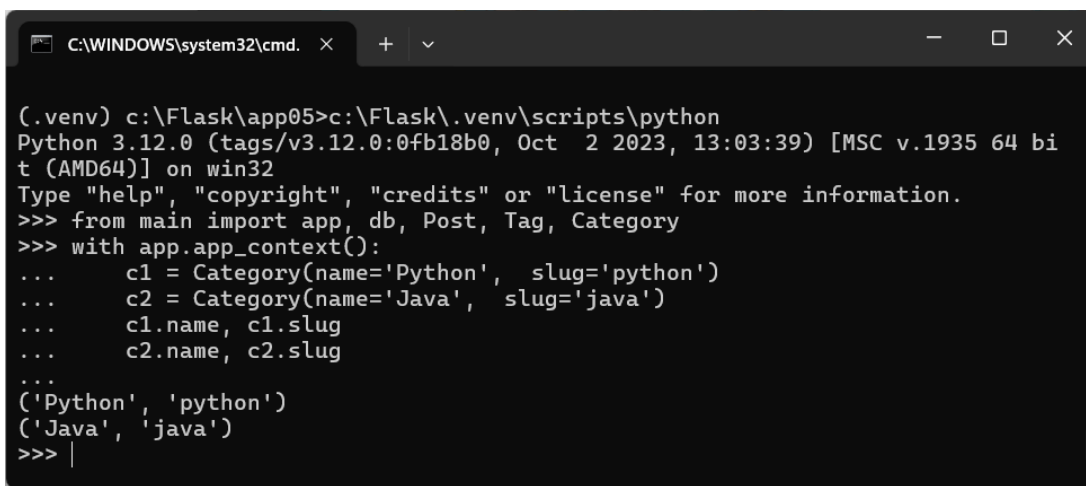
2.1 Додавання даних

Щоб створити новий запис із даними за допомогою SQLAlchemy, потрібно виконати такі кроки:

1. Створити об'єкт
2. Додати об'єкт у сесію
3. Завантажити (зробити коміт) сесію

У SQLAlchemy взаємодія з базою даних відбувається за допомогою сесії. На щастя, її не потрібно створювати вручну. Це робить Flask-SQLAlchemy. Доступ до об'єкта сесії можна отримати за допомогою `db.session`. Це об'єкт сесії, який відповідає за підключення до бази даних. Він же відповідає за процес транзакції. За замовчуванням транзакція запускається і залишається відкритою доти, доки виконуються коміти і відкати.

Запустимо оболонку Python для створення деяких об'єктів моделі:

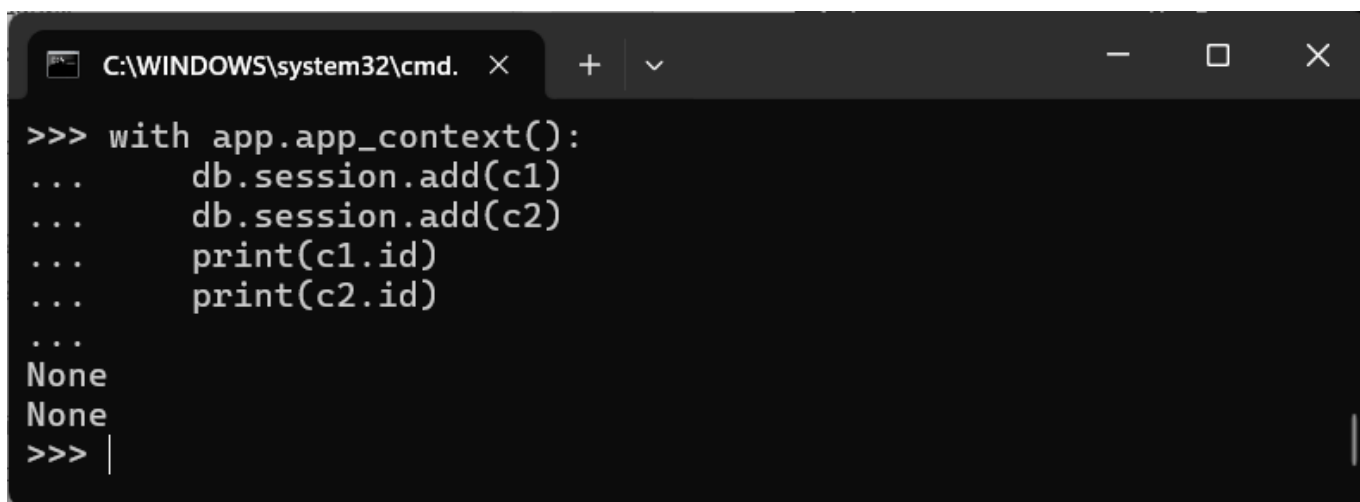


```
(.venv) c:\Flask\app05>c:\Flask\.venv\scripts\python
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct 2 2023, 13:03:39) [MSC v.1935 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> from main import app, db, Post, Tag, Category
>>> with app.app_context():
...     c1 = Category(name='Python', slug='python')
...     c2 = Category(name='Java', slug='java')
...     c1.name, c1.slug
...     c2.name, c2.slug
...
('Python', 'python')
('Java', 'java')
>>> |
```

Було

створено два об'єкти *Category*. Отримати доступ до їхніх атрибутів можна за допомогою оператора крапки (.).

Далі необхідно додати об'єкти в сесію.



```

C:\WINDOWS\system32\cmd. x + v
>>> with app.app_context():
...     db.session.add(c1)
...     db.session.add(c2)
...     print(c1.id)
...     print(c2.id)
...
None
None
>>> |

```

Додавання об'єктів не записує їх у базу даних, цей процес лише готує їх для збереження під час наступної комітації. Упевнитися в цьому можна, перевіривши первинний ключ об'єктів.

Значення атрибута `id` обох об'єктів - `None`. Це означає, що об'єкти не збережені в базі даних.

Замість додавання по одному об'єкту в сесію щоразу, можна використовувати метод `add_all()`. Метод `add_all()` приймає список об'єктів, які потрібно додати в сесію.

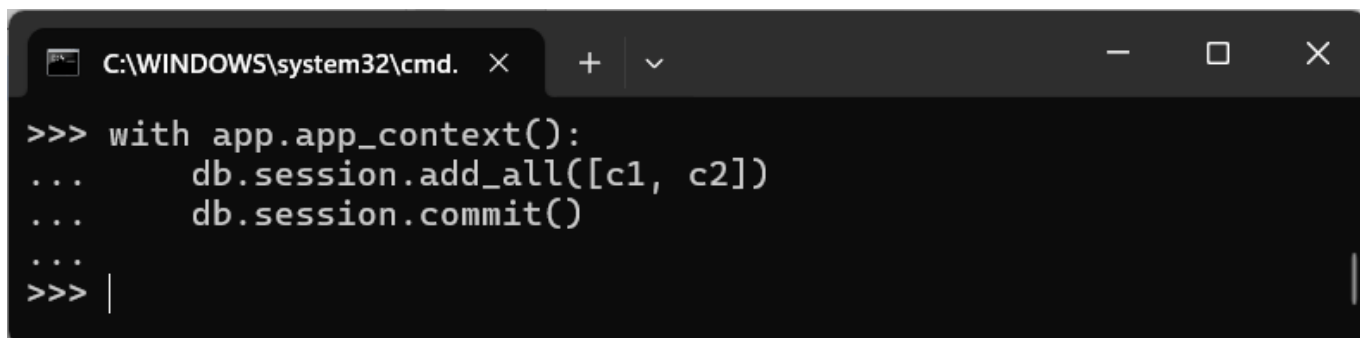
```

with app.app_context():
    db.session.add_all([c1, c2])
    db.session.new
    db.session.commit()

```

Якщо спробувати додати об'єкт у сесію кілька разів, помилок не виникне.

У будь-який момент можна подивитися всі об'єкти сесії за допомогою `db.session.new`. Нарешті, для збереження об'єктів у базі даних потрібно викликати метод `commit()`:

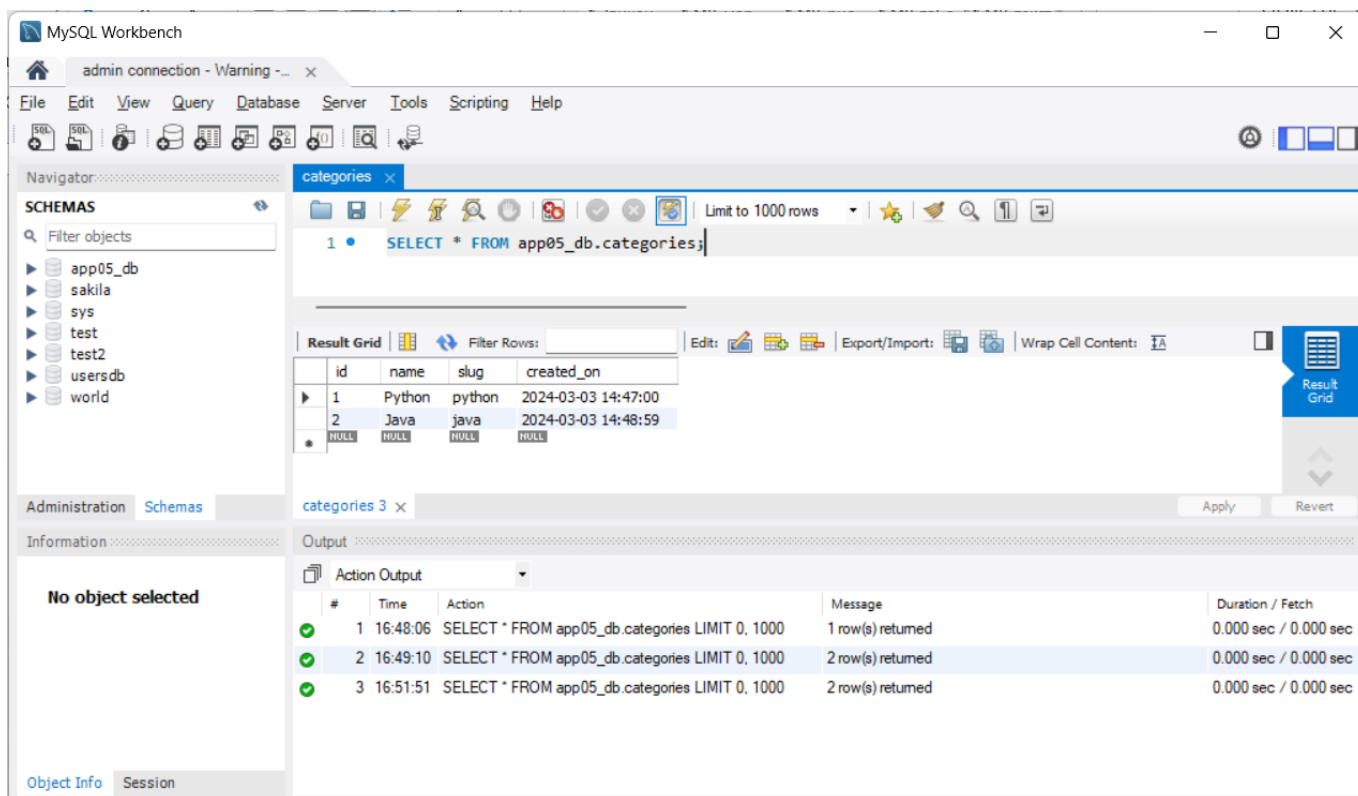


```

C:\WINDOWS\system32\cmd. x + v
>>> with app.app_context():
...     db.session.add_all([c1, c2])
...     db.session.commit()
...
>>> |

```

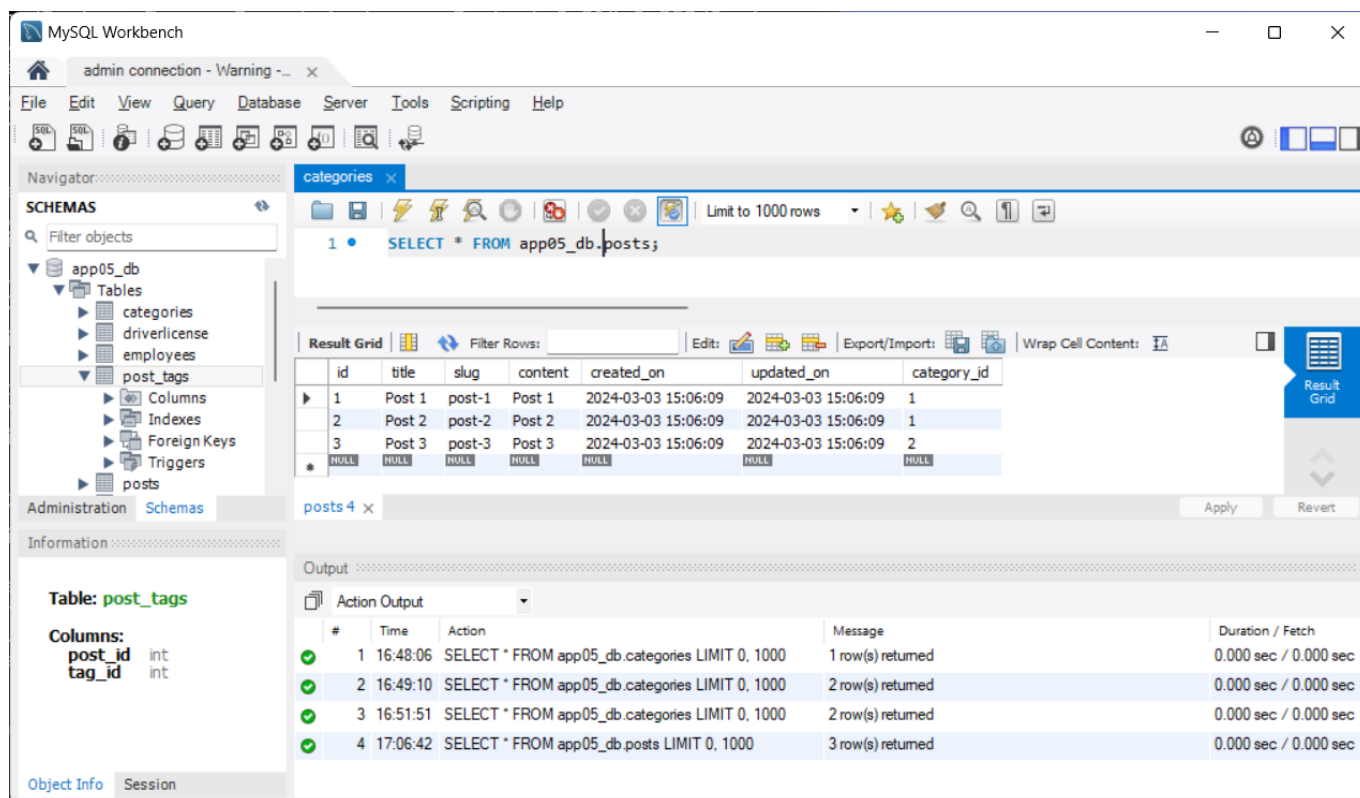
На цьому етапі таблиця *categories* має виглядати приблизно так:



Нові категорії поки не пов'язані з постами. Тому *c1.posts* і *c2.posts* повернуть порожній список.

Варто спробувати створити кілька постів. Далі потрібно додати об'єкти в сесію і зробити коміт.

```
C:\WINDOWS\system32\cmd. x + v
>>> with app.app_context():
...     p1 = Post(title='Post 1', slug='post-1', content='Post 1', category=c1)
...     p2 = Post(title='Post 2', slug='post-2', content='Post 2', category=c1)
...     p3 = Post(title='Post 3', slug='post-3', content='Post 3', category=c2)
...     db.session.add_all([p1, p2, p3])
...     db.session.commit()
...
>>> |
```



Якщо зараз спробувати отримати доступ до атрибута *posts* об'єкта *Category*, то він поверне не-порожній список:

```
>>>
>>> c1.posts
[<1:Post 1>, <2:Post 2>]
>>>
>>> c2.posts
[<3:Post 3>]
>>>
```

З іншого боку відношення, можна отримати доступ до об'єкта *Category*, до якого належить пост, за допомогою атрибута *category* в об'єкта *Post*.

```
>>>
>>> p1.category
<1:Python>
>>>
>>> p2.category
<1:Python>
>>>
>>> p3.category
<2:Java>
>>>
```

Варто нагадати, що все це можливо завдяки інструкції *relationship()* у моделі *Category*. Зараз у базі даних є три пости, але жоден із них не пов'язаний із тегами.

```
>>>
>>> p1.tags, p2.tags, p3.tags
([], [], [])
>>>
```

Настав час створити теги. Це можна зробити в оболонці таким чином:

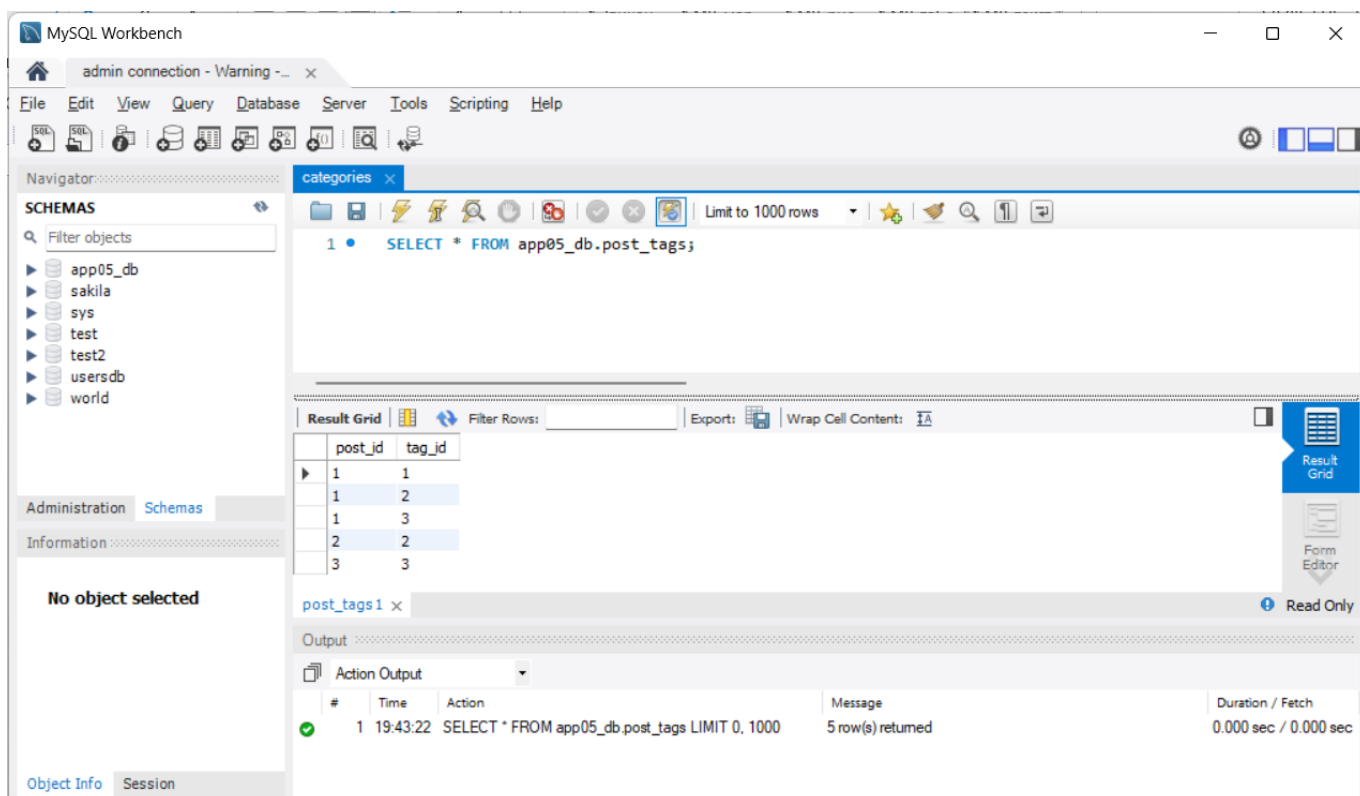
```
with app.app_context():
    t1 = Tag(name="refactoring", slug="refactoring")
    t2 = Tag(name="snippet", slug="snippet")
    t3 = Tag(name="analytics", slug="analytics")

    db.session.add_all([t1, t2, t3])
    db.session.commit()
```

Цей код створює три об'єкти тегів і робить їхній коміт у базу даних. Пости все ще не прив'язані до тегів. Ось як можна пов'язати об'єкт *Post* з об'єктом *Tag*.

```
with app.app_context():
    p1.tags.append(t1)
    p1.tags.extend([t2, t3])
    p2.tags.append(t2)
    p3.tags.append(t3)
    db.session.add_all([p1, p2, p3])
    db.session.commit()
```

Цей коміт додає наступні п'ять записів у таблицю *post_tags*.



Повний перелік команд наступний:

```

C:\WINDOWS\system32\cmd. x + v
>>> from main import app, db, Post, Tag, Category
>>> with app.app_context():
...     c1 = Category(name='Python', slug='python')
...     c2 = Category(name='Java', slug='java')
...     db.session.add_all([c1, c2])
...     db.session.commit()
...     p1 = Post(title='Post 1', slug='post-1', content='Post 1', category=c1)
...     p2 = Post(title='Post 2', slug='post-2', content='Post 2', category=c1)
...     p3 = Post(title='Post 3', slug='post-3', content='Post 3', category=c2)
...     db.session.add_all([p1, p2, p3])
...     db.session.commit()
...     t1 = Tag(name='refactoring', slug='refactoring')
...     t2 = Tag(name='snippet', slug='snippet')
...     t3 = Tag(name='analytics', slug='analytics')
...     db.session.add_all([t1, t2, t3])
...     db.session.commit()
...     p1.tags.append(t1)
...     p1.tags.extend([t2, t3])
...     p2.tags.append(t2)
...     p3.tags.append(t3)
...     db.session.add_all([p1, p2, p3])
...     db.session.commit()
>>>

```

Пости тепер пов'язані з одним або більшою кількістю тегів:

Барбарук Віктор Миколайович. Сучасні технології розробки WEB-застосунків з використанням мови Python


```

>>>
>>> p1.tags
[<1:refactoring>, <2:snippet>, <3:analytics>]
>>>
>>> p2.tags
[<2:snippet>]
>>>
>>> p3.tags
[<3:analytics>]
>>>

```

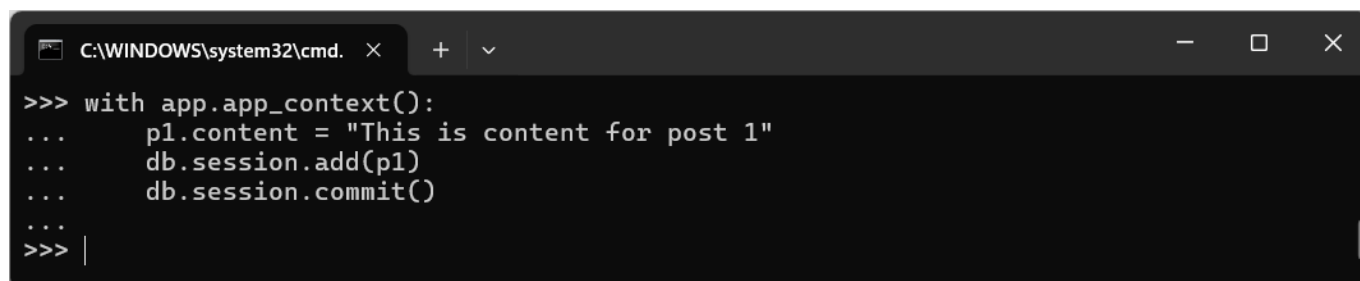
З іншого боку можна отримати доступ до постів, які відносяться до конкретного тегу:

```

>>>
>>> t1.posts
[<1:Post 1>]
>>>
>>> t2.posts
[<1:Post 1>, <2:Post 2>]
>>>
>>> t3.posts
[<1:Post 1>, <3:Post 3>]
>>>
>>>

```

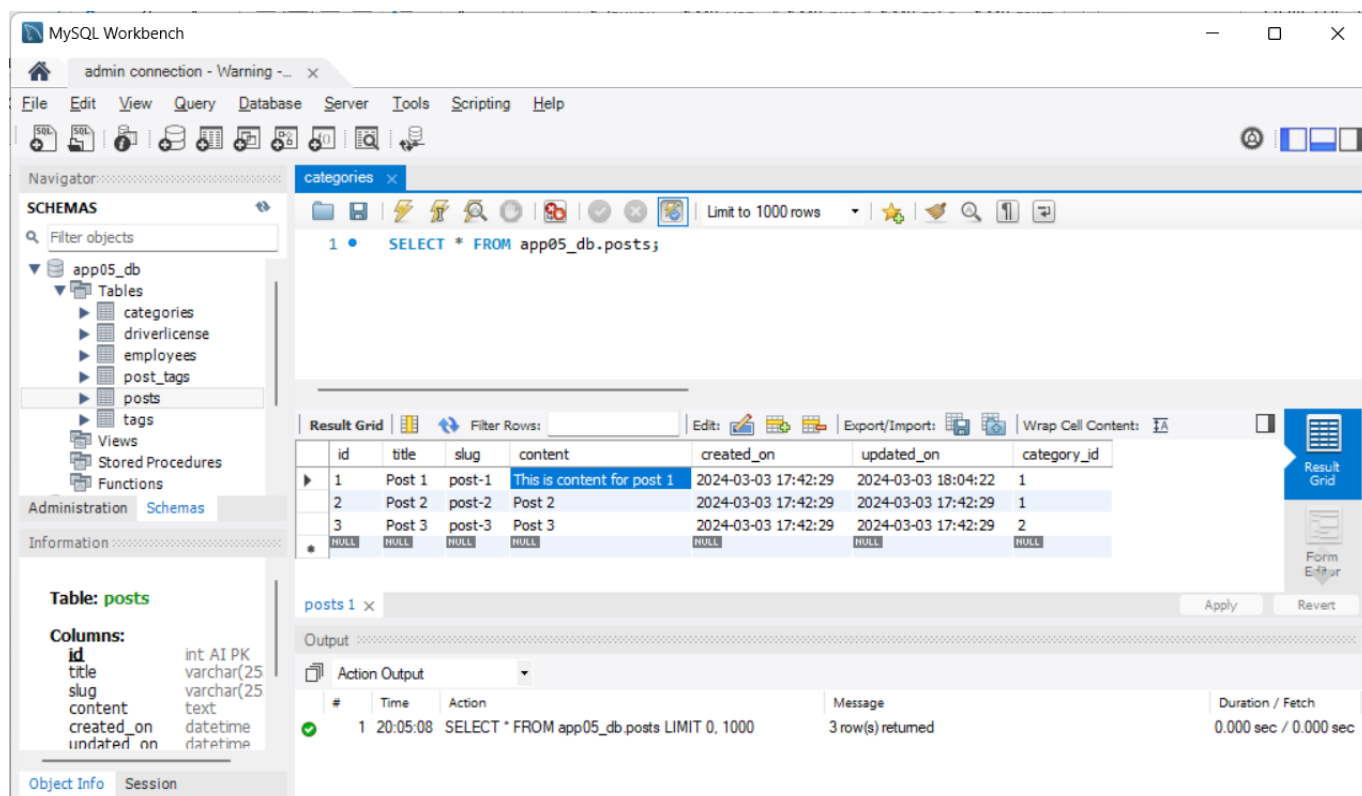
Для оновлення об'єкта потрібно всього лише передати його атрибуту нове значення, додати об'єкт у сесію і зробити коміт.



```

C:\WINDOWS\system32\cmd. x + v
>>> with app.app_context():
...     p1.content = "This is content for post 1"
...     db.session.add(p1)
...     db.session.commit()
...
>>> |

```



2.2 Видалення даних

Для видалення об'єкта потрібно використовувати метод *delete()* об'єкта сесії. Він приймає об'єкт і зазначає, що той підлягає видаленню під час наступного коміту.

Створимо новий тимчасовий тег *seo* і зв'яжемо його з постами *p1* і *p2*:

```
with app.app_context():
    tmp = Tag(name='seo', slug='seo')
    p1.tags.append(tmp)
    p2.tags.append(tmp)
    db.session.add_all([p1, p2])
    db.session.commit()
```

Цей коміт додає всього 3 рядки: один у таблицю *tags* і ще два - у таблицю *post_tags*. У базі даних ці три рядки мають такий вигляд:

MySQL Workbench

admin connection - Warning - ...

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

app05_db

Tables

- categories
- driverlicense
- employees
- post_tags
- posts
- tags

Columns

Indexes

Foreign Keys

Administration Schemas

Information

Table: tags

Columns:

- id int AI PK
- name varchar(255)
- slug varchar(255)
- created_on datetime

Object Info Session

categories tags

Limit to 1000 rows

1 • SELECT * FROM app05_db.tags;

Result Grid

	id	name	slug	created_on
1	refactoring	refactoring	2024-03-03 19:45:01	
2	snippet	snippet	2024-03-03 19:45:01	
3	analytics	analytics	2024-03-03 19:45:01	
4	seo	seo	2024-03-04 10:38:11	

tags 1 x

Apply Revert

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:47:37	SELECT * FROM app05_db.tags LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec

MySQL Workbench

admin connection - Warning - ...

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

app05_db

Tables

- categories
- driverlicense
- employees
- post_tags
- posts
- tags

Columns

Indexes

Foreign Keys

Triggers

Views

Stored Procedures

Administration Schemas

Information

Table: tags

Columns:

- id int AI PK
- name varchar(255)
- slug varchar(255)
- created_on datetime

Object Info Session

categories tags post_tags

Limit to 1000 rows

1 • SELECT * FROM app05_db.post_tags;

Result Grid

	post_id	tag_id
1	1	1
1	1	2
1	1	3
2	2	2
3	3	3
1	4	4
2	4	4

post_tags 2 x

Read Only

Output

Action Output

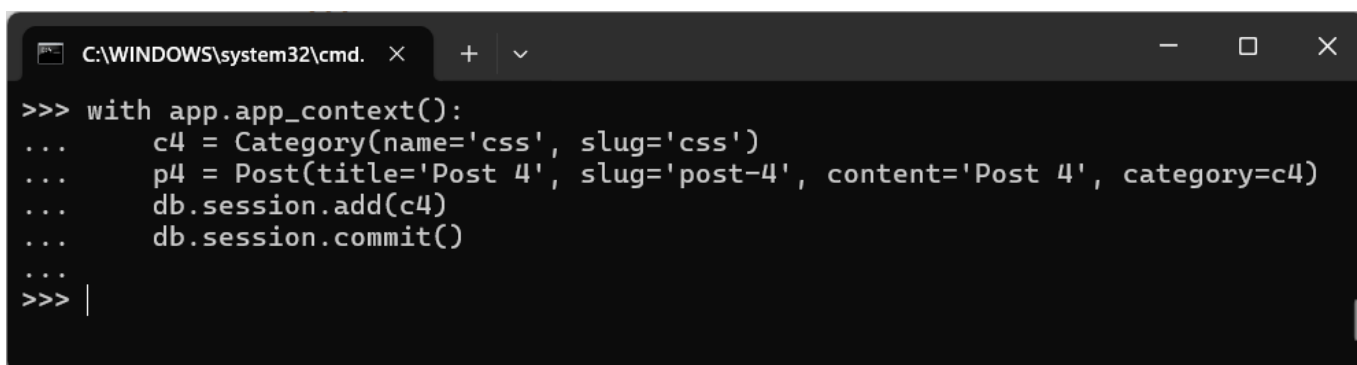
#	Time	Action	Message	Duration / Fetch
2	12:48:43	SELECT * FROM app05_db.post_tags LIMIT 0, 1000	7 row(s) returned	0.000 sec / 0.000 sec
3	12:49:01	SELECT * FROM app05_db.post_tags LIMIT 0, 1000	7 row(s) returned	0.000 sec / 0.000 sec

Тепер потрібно видалити тег *seo*:

```
with app.app_context():
    db.session.delete(tmp)
    db.session.commit()
```

Цей коміт видаляє всі три рядки, додані в попередньому кроці. Проте він не видаляє пост, з яким тег був пов'язаний.

За замовчуванням під час видалення об'єкта в батьківській таблиці (наприклад, *categories*) значення зовнішнього ключа об'єкта, який із ним пов'язаний у дочірній таблиці (наприклад, *posts*) стає NULL. Наступний код демонструє цю поведінку на прикладі створення нового об'єкта категорії та об'єкта поста, який з нею пов'язаний, і подальшим видаленням об'єкта категорії:



```
C:\WINDOWS\system32\cmd.  X  +  v  -  □  X

>>> with app.app_context():
...     c4 = Category(name='css', slug='css')
...     p4 = Post(title='Post 4', slug='post-4', content='Post 4', category=c4)
...     db.session.add(c4)
...     db.session.commit()
...
>>> |
```

Цей коміт додає два рядки. Один у таблицю *categories*, і ще один - у таблицю *posts*.

MySQL Workbench

admin connection - Warning - ...

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

app05_db

- Tables
 - categories
 - driverlicense
 - employees
 - post_tags
 - posts
 - tags
- Columns
- Indexes
- Foreign Keys
- Triggers
- Views
- Stored Procedures

Administration Schemas

Information

Table: tags

Columns:

- id int AI PK
- name varchar(255)
- slug varchar(255)
- created_on datetime

Object Info Session

tags post_tags categories

Limit to 1000 rows

1 • SELECT * FROM app05_db.categories;

Result Grid

	id	name	slug	created_on
1	1	Python	python	2024-03-04 10:38:11
2	2	Java	java	2024-03-04 10:38:11
3	3	CSS	CSS	2024-03-04 10:59:58
*	NULL	NULL	NULL	NULL

categories 2 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
6	13:04:47	SELECT * FROM app05_db.categories LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
7	13:04:56	SELECT * FROM app05_db.categories LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec

MySQL Workbench

admin connection - Warning - ...

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

app05_db

- Tables
 - categories
 - driverlicense
 - employees
 - post_tags
 - posts
 - tags
- Columns
- Indexes
- Foreign Keys
- Triggers
- Views
- Stored Procedures
- Functions

Administration Schemas

Information

Table: tags

Columns:

- id int AI PK
- name varchar(255)
- slug varchar(255)
- created_on datetime

Object Info Session

tags post_tags categories posts

Limit to 1000 rows

1 • SELECT * FROM app05_db.posts;

Result Grid

	id	title	slug	content	created_on	updated_on	category_id
1	1	Post 1	post-1	Post 1	2024-03-04 10:38:11	2024-03-04 10:38:11	1
2	2	Post 2	post-2	Post 2	2024-03-04 10:38:11	2024-03-04 10:38:11	1
3	3	Post 3	post-3	Post 3	2024-03-04 10:38:11	2024-03-04 10:38:11	2
4	4	Post 4	post-4	Post 4	2024-03-04 10:59:58	2024-03-04 10:59:58	3
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

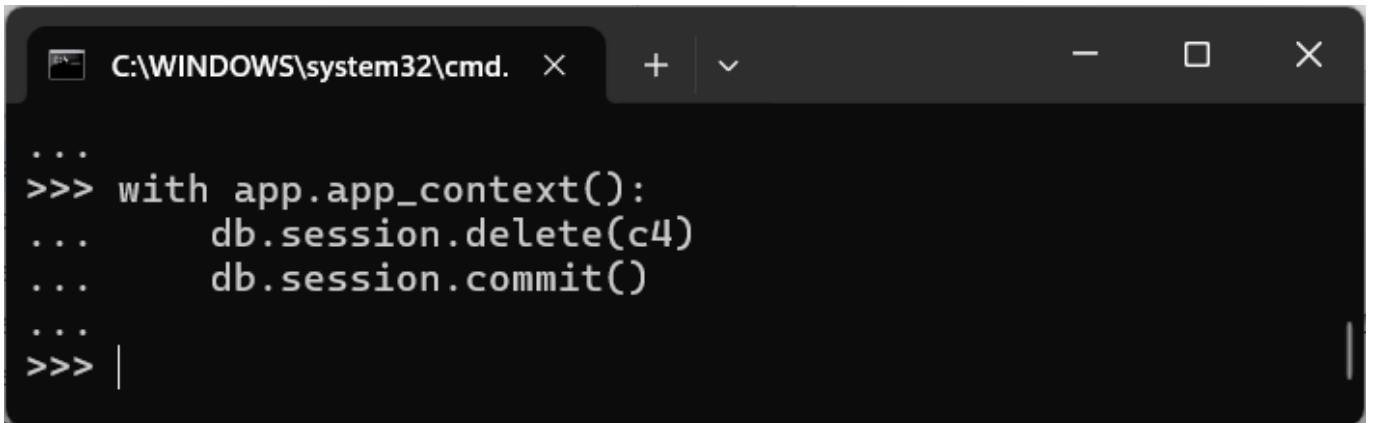
posts 1 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
7	13:04:56	SELECT * FROM app05_db.categories LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
8	13:06:16	SELECT * FROM app05_db.posts LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec

Тепер потрібно подивитися, що відбувається під час видалення об'єкта *Category*.

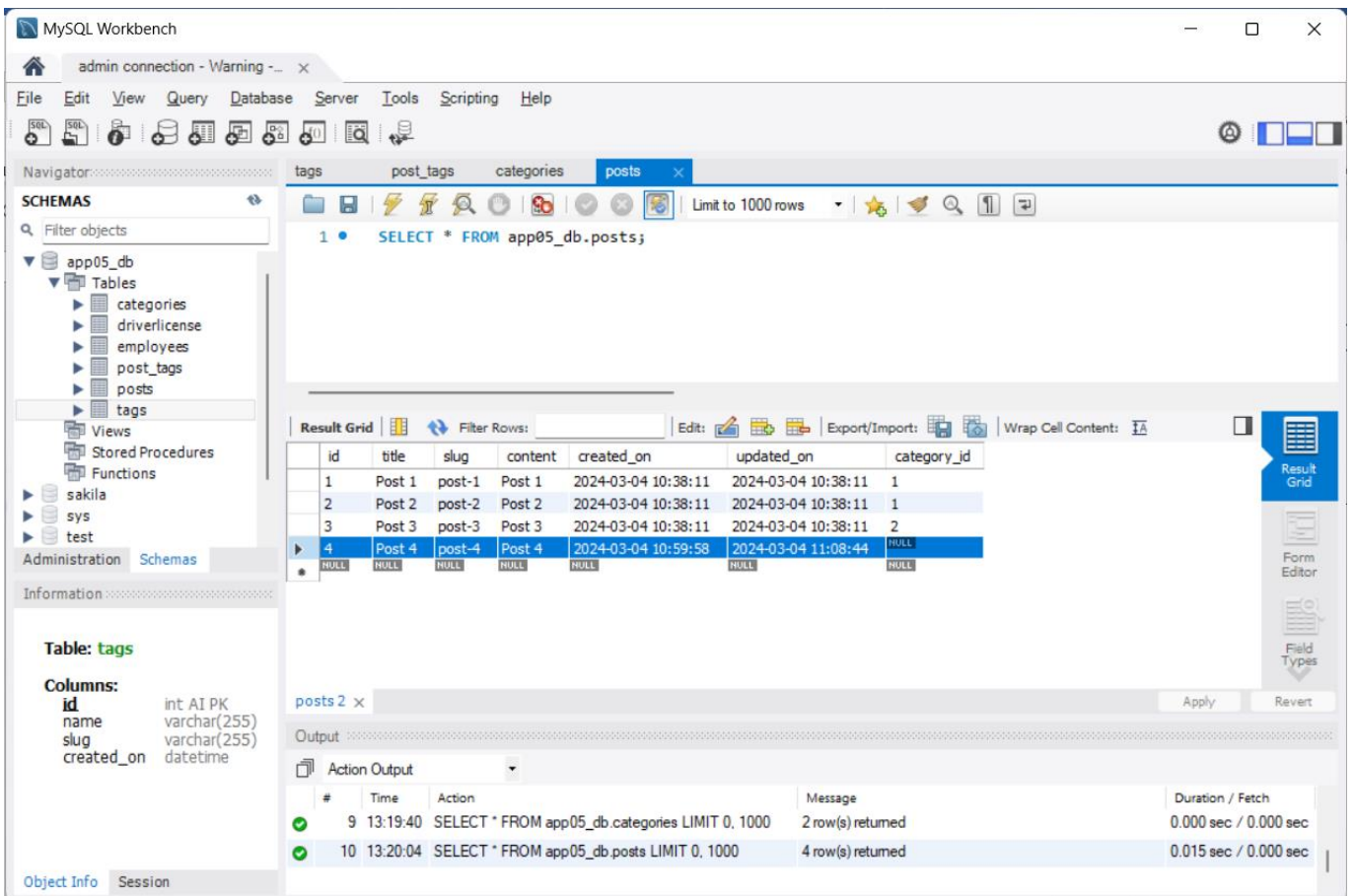


```

C:\WINDOWS\system32\cmd.
...
>>> with app.app_context():
...     db.session.delete(c4)
...     db.session.commit()
...
>>> |

```

Цей коміт видаляє категорію *css* з таблиці *categories* і встановлює значення зовнішнього ключа (*category_id*) для поста, який з нею пов'язаний, на *NULL*.

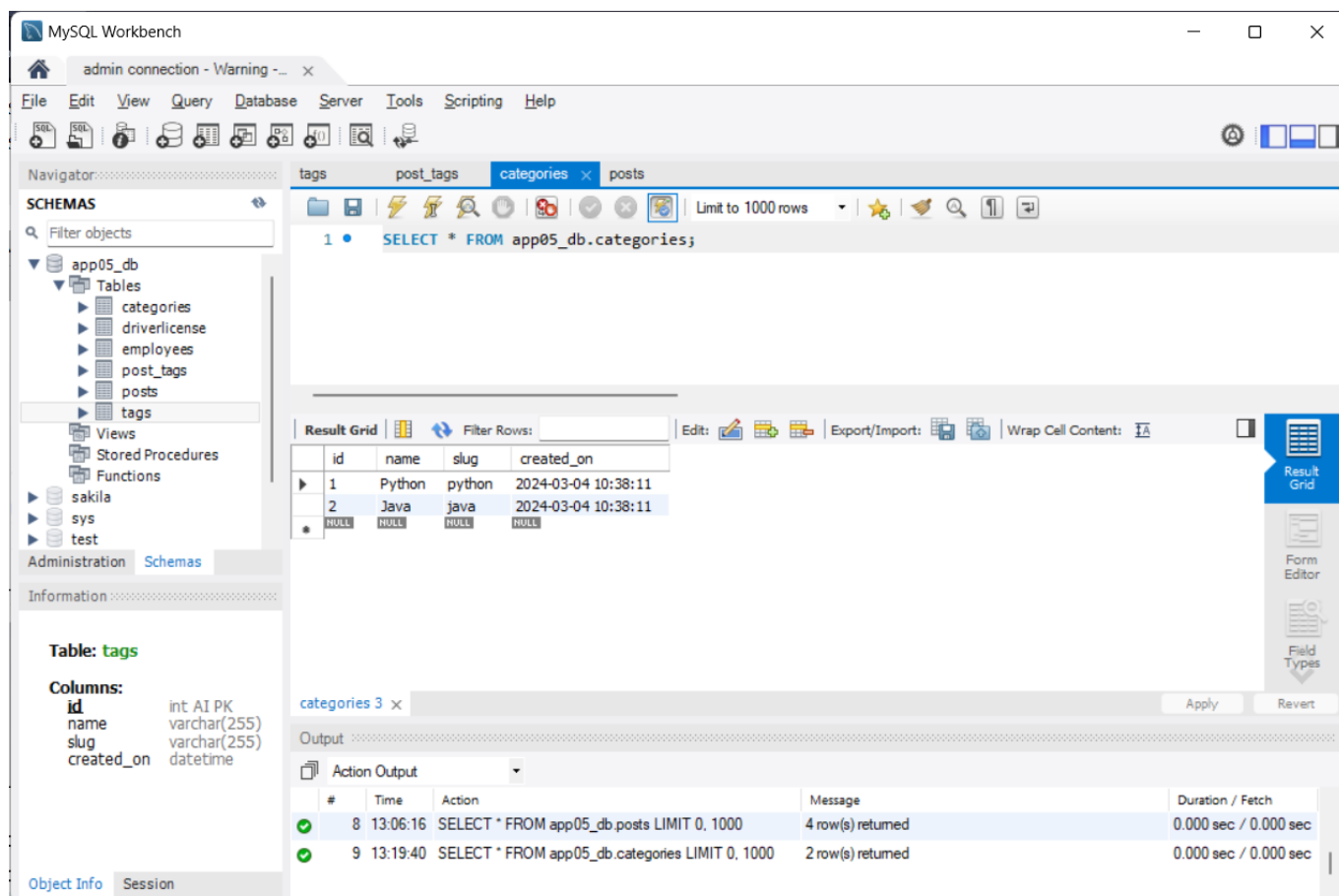


The screenshot shows the MySQL Workbench interface. The 'posts' table is selected in the Navigator. The SQL query editor contains the query: `SELECT * FROM app05_db.posts;`. The Result Grid shows the following data:

#	id	title	slug	content	created_on	updated_on	category_id
1	Post 1	post-1	Post 1	2024-03-04 10:38:11	2024-03-04 10:38:11	1	
2	Post 2	post-2	Post 2	2024-03-04 10:38:11	2024-03-04 10:38:11	1	
3	Post 3	post-3	Post 3	2024-03-04 10:38:11	2024-03-04 10:38:11	2	
4	Post 4	post-4	Post 4	2024-03-04 10:59:58	2024-03-04 11:08:44	NULL	

The 'category_id' for the fourth row is NULL. The bottom panel shows the 'Action Output' with the following messages:

#	Time	Action	Message	Duration / Fetch
9	13:19:40	SELECT * FROM app05_db.categories LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
10	13:20:04	SELECT * FROM app05_db.posts LIMIT 0, 1000	4 row(s) returned	0.015 sec / 0.000 sec



У деяких випадках може виникнути необхідність видалити всі дочірні записи при тому, що батьківські записи вже видалено. Це можна зробити, передавши *cascade='all,delete-orphan'* інструкції *db.relationship()*. Відкриємо *main.py*, щоб змінити інструкцію *db.relationship()* у моделі *Category*:

```
class Category(db.Model):
    __tablename__ = 'categories'
    id = db.Column(db.Integer(), primary_key=True)
    name = db.Column(db.String(255), nullable=False)
    slug = db.Column(db.String(255), nullable=False)
    created_on = db.Column(db.DateTime(), default=datetime.utcnow)
    posts = db.relationship('Post', backref='category', cascade='all,delete-orphan')

    def __repr__(self):
        return "<{}:{}]>".format(id, self.name)
```

З цього моменту видалення категорії спричинить видалення постів, які з нею пов'язані. Щоб це почало працювати, потрібно перезапустити оболонку. Далі імпортуємо потрібні об'єкти та створюємо категорію разом із постом:

```

C:\WINDOWS\system32\cmd. x + v
(.venv) c:\Flask\app05>c:\Flask\.venv\scripts\python
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct 2 2023, 13:03:39) [MSC v.1935 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> from main import app, db, Post, Tag, Category
>>> with app.app_context():
...     c5 = Category(name='css', slug='css')
...     p5 = Post(title='Post 5', slug='post-5', content='Post 5', category=c5)
...     db.session.add(c5)
...     db.session.commit()
...
>>> |

```

Ось який вигляд база даних має після цього коміту.

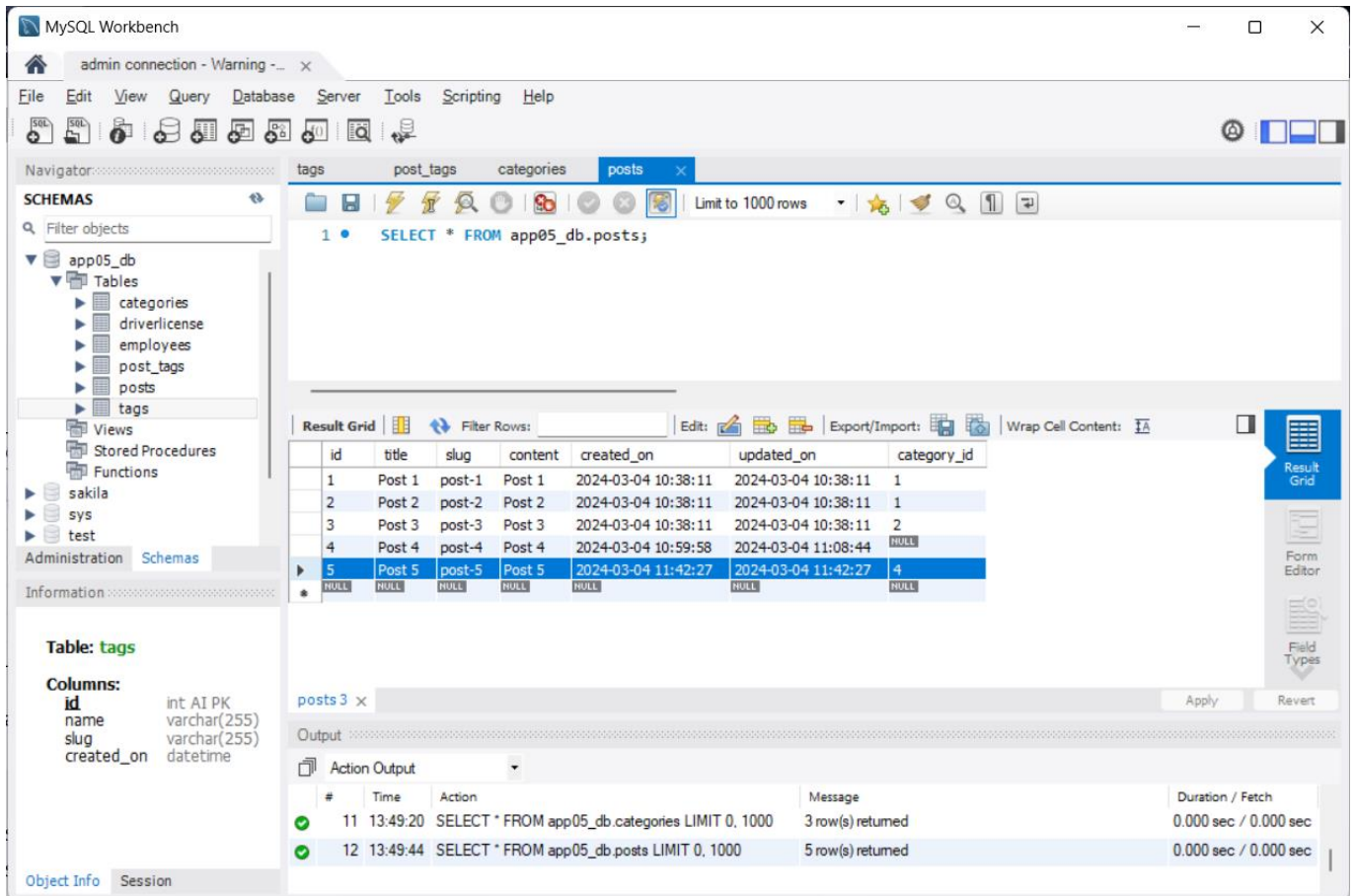
The screenshot displays the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows the 'app05_db' database with tables 'categories', 'driverlicense', 'employees', 'post_tags', 'posts', and 'tags'. The 'categories' table is selected. The main window shows the 'categories' table with the following data:

id	name	slug	created_on
1	Python	python	2024-03-04 10:38:11
2	Java	java	2024-03-04 10:38:11
4	css	css	2024-03-04 11:42:27
HULL	HULL	HULL	HULL

The 'Information' pane shows the table structure:

Column	Type	Attributes
id	int	AI, PK
name	varchar(255)	
slug	varchar(255)	
created_on	datetime	

The 'Output' pane shows the execution of a query: 'SELECT * FROM app05_db.categories LIMIT 0, 1000' with 3 rows returned.



Видалимо категорію:

```

C:\WINDOWS\system32\cmd.
>>> with app.app_context():
...     db.session.delete(c5)
...     db.session.commit()
>>>

```

Після цього комміту база даних має ось такий вигляд:

MySQL Workbench

admin connection - Warning - ...

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

app05_db

- Tables
 - categories
 - driverlicense
 - employees
 - post_tags
 - posts
 - tags
- Views
- Stored Procedures
- Functions
- sakila
- sys
- test

Administration Schemas

Information

Table: tags

Columns:

- id int AI PK
- name varchar(255)
- slug varchar(255)
- created_on datetime

tags post_tags categories posts

Limit to 1000 rows

1 • `SELECT * FROM app05_db.categories;`

Result Grid

	id	name	slug	created_on
▶	1	Python	python	2024-03-04 10:38:11
▶	2	Java	java	2024-03-04 10:38:11
*	NULL	NULL	NULL	NULL

categories 5 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓	12 13:49:44	SELECT * FROM app05_db.posts LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
✓	13 14:04:13	SELECT * FROM app05_db.categories LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

MySQL Workbench

admin connection - Warning - ...

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

app05_db

- Tables
 - categories
 - driverlicense
 - employees
 - post_tags
 - posts
 - tags
- Views
- Stored Procedures
- Functions
- sakila
- sys
- test

Administration Schemas

Information

Table: tags

Columns:

- id int AI PK
- name varchar(255)
- slug varchar(255)
- created_on datetime

tags post_tags categories posts

Limit to 1000 rows

1 • `SELECT * FROM app05_db.posts;`

Result Grid

	id	title	slug	content	created_on	updated_on	category_id
▶	1	Post 1	post-1	Post 1	2024-03-04 10:38:11	2024-03-04 10:38:11	1
▶	2	Post 2	post-2	Post 2	2024-03-04 10:38:11	2024-03-04 10:38:11	1
▶	3	Post 3	post-3	Post 3	2024-03-04 10:38:11	2024-03-04 10:38:11	2
▶	4	Post 4	post-4	Post 4	2024-03-04 10:59:58	2024-03-04 11:08:44	NULL
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

posts 4 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓	13 14:04:13	SELECT * FROM app05_db.categories LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
✓	14 14:04:36	SELECT * FROM app05_db.posts LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

2.3 Запит даних

Щоб виконати запит до бази даних, використовується метод *query()* об'єкта *session*. Метод *query()* повертає об'єкт *flask_sqlalchemy.BaseQuery*, який є розширенням оригінального об'єкта *sqlalchemy.orm.query.Query.Query*. Об'єкт *flask_sqlalchemy.BaseQuery* являє собою оператор *SELECT*, який буде використовуватися для здійснення запитів до бази даних. У цій таблиці перелічено основні методи класу *flask_sqlalchemy.BaseQuery*.

Метод	Опис
<i>all()</i>	Повертає результат запиту (представлений <i>flask_sqlalchemy.BaseQuery</i>) у вигляді списку.
<i>count()</i>	Повертає кількість записів у запиті.
<i>first()</i>	Повертає перший результат запиту або <i>None</i> , якщо в ньому немає рядків.
<i>first_or_404()</i>	Повертає перший результат запиту або помилку 404, якщо в ньому немає рядків.
<i>get(pk)</i>	Повертає об'єкт, який відповідає даному первинному ключу або <i>None</i> , якщо об'єкт не знайдено.
<i>get_or_404(pk)</i>	Повертає об'єкт, який відповідає даному первинному ключу або помилку 404, якщо об'єкт не знайдено.
<i>filter(*criterion)</i>	Повертає новий екземпляр <i>flask_sqlalchemy.BaseQuery</i> з оператором <i>WHERE</i> .
<i>limit(limit)</i>	Повертає новий екземпляр <i>flask_sqlalchemy.BaseQuery</i> з оператором <i>LIMIT</i> .
<i>offset(offset)</i>	Повертає новий екземпляр <i>flask_sqlalchemy.BaseQuery</i> з оператором <i>OFFSET</i> .
<i>order_by(*criterion)</i>	Повертає новий екземпляр <i>flask_sqlalchemy.BaseQuery</i> з оператором <i>OFFSET</i> .
<i>join()</i>	Повертає новий екземпляр <i>flask_sqlalchemy.BaseQuery</i> після створення SQL <i>JOIN</i> .

2.3.1 Метод *all()*

У своїй найпростішій формі метод *query()* приймає як аргументи один або більше класів моделі або колонки. Наступний код поверне всі записи з таблиці *posts*.

```

C:\WINDOWS\system32\cmd. x + v - □ x

...
>>> with app.app_context():
...     db.session.query(Post).all()
...
[<1:Post 1>, <2:Post 2>, <3:Post 3>, <4:Post 4>]
>>> |

```

Схожим чином наступний код поверне всі записи з таблиць *categories* і *tags*.

```

C:\WINDOWS\system32\cmd. x + v - □ x

...
>>> with app.app_context():
...     db.session.query(Post).all()
...
[<1:Post 1>, <2:Post 2>, <3:Post 3>, <4:Post 4>]
>>> with app.app_context():
...     db.session.query(Category).all()
...     db.session.query(Tag).all()
...
[<<built-in function id>:Python>, <<built-in function id>:Java>]
[<<built-in function id>:refactoring>, <<built-in function id>:snippet>, <<built
-in function id>:analytics>]
>>> |

```

Щоб отримати чистий SQL, використаний для запиту до бази даних, потрібно просто вивести об'єкт *flask_sqlalchemy.BaseQuery*:

```

C:\WINDOWS\system32\cmd. x + v - □ x

>>> with app.app_context():
...     print(db.session.query(Post), '\n')
...
SELECT posts.id AS posts_id, posts.title AS posts_title, posts.slug AS posts_slug, posts.content AS posts_content, posts.created_on AS posts_created_on, posts.updated_on AS posts_updated_on, posts.category_id AS posts_category_id
FROM posts
>>> |

```

У попередніх прикладах дані поверталися з усіх колонок таблиці. Це можна змінити, передавши методу *query()* назви колонок:

```
C:\WINDOWS\system32\cmd. x + v
>>> with app.app_context():
...     db.session.query(Post.id, Post.title).all()
...
[(1, 'Post 1'), (2, 'Post 2'), (3, 'Post 3'), (4, 'Post 4')]
>>> |
```

2.3.2 Метод `count()`

Метод *count()* повертає кількість результатів у запиті.

```
C:\WINDOWS\system32\cmd. x + v
>>> with app.app_context():
...     db.session.query(Post).count()
...     db.session.query(Category).count()
...     db.session.query(Tag).count()
...
4
2
3
>>> |
```

2.3.3. Метод `first()`

Метод *first()* поверне тільки перший запит із запиту або *None*, якщо в запиті немає результатів.

```

C:\WINDOWS\system32\cmd. x + v
>>> with app.app_context():
...     db.session.query(Post).first()
...     db.session.query(Category).first()
...     db.session.query(Tag).first()
...
<1:Post 1>
<<built-in function id>:Python>
<<built-in function id>:refactoring>
>>> |

```

2.3.4 Метод get()

Метод *get()* поверне екземпляр об'єкта з відповідним первинним ключем або *None*, якщо такий об'єкт не було знайдено.

```

C:\WINDOWS\system32\cmd. x + v
>>> with app.app_context():
...     db.session.query(Post).get(2)
...     db.session.query(Category).get(1)
...     print(db.session.query(Category).get(10))
...
<stdin>:2: LegacyAPIWarning: The Query.get() method is considered legacy
as of the 1.x series of SQLAlchemy and becomes a legacy construct in 2.0.
The method is now available as Session.get() (deprecated since: 2.0) (Ba
ckground on SQLAlchemy 2.0 at: https://sqlalche.me/e/b8d9)
<2:Post 2>
<stdin>:3: LegacyAPIWarning: The Query.get() method is considered legacy
as of the 1.x series of SQLAlchemy and becomes a legacy construct in 2.0.
The method is now available as Session.get() (deprecated since: 2.0) (Ba
ckground on SQLAlchemy 2.0 at: https://sqlalche.me/e/b8d9)
<<built-in function id>:Python>
<stdin>:4: LegacyAPIWarning: The Query.get() method is considered legacy
as of the 1.x series of SQLAlchemy and becomes a legacy construct in 2.0.
The method is now available as Session.get() (deprecated since: 2.0) (Ba
ckground on SQLAlchemy 2.0 at: https://sqlalche.me/e/b8d9)
None
>>> |

```

2.3.5 Метод `get_or_404()`

```
C:\WINDOWS\system32\cmd. x + v
>>> with app.app_context():
...     db.session.query(Post).get_or_404(1)
...     db.session.query(Post).get_or_404(100)
...
<1:Post 1>
Traceback (most recent call last):
  File "<stdin>", line 3, in <module>
  File "c:\Flask\.venv\Lib\site-packages\flask_sqlalchemy\query.py", line
  33, in get_or_404
    abort(404, description=description)
  File "c:\Flask\.venv\Lib\site-packages\flask\helpers.py", line 272, in
  abort
    current_app.aborter(code, *args, **kwargs)
  File "c:\Flask\.venv\Lib\site-packages\werkzeug\exceptions.py", line 86
  1, in __call__
    raise self.mapping[code](*args, **kwargs)
werkzeug.exceptions.NotFound: 404 Not Found: The requested URL was not fo
und on the server. If you entered the URL manually please check your spel
ling and try again.
>>> |
```

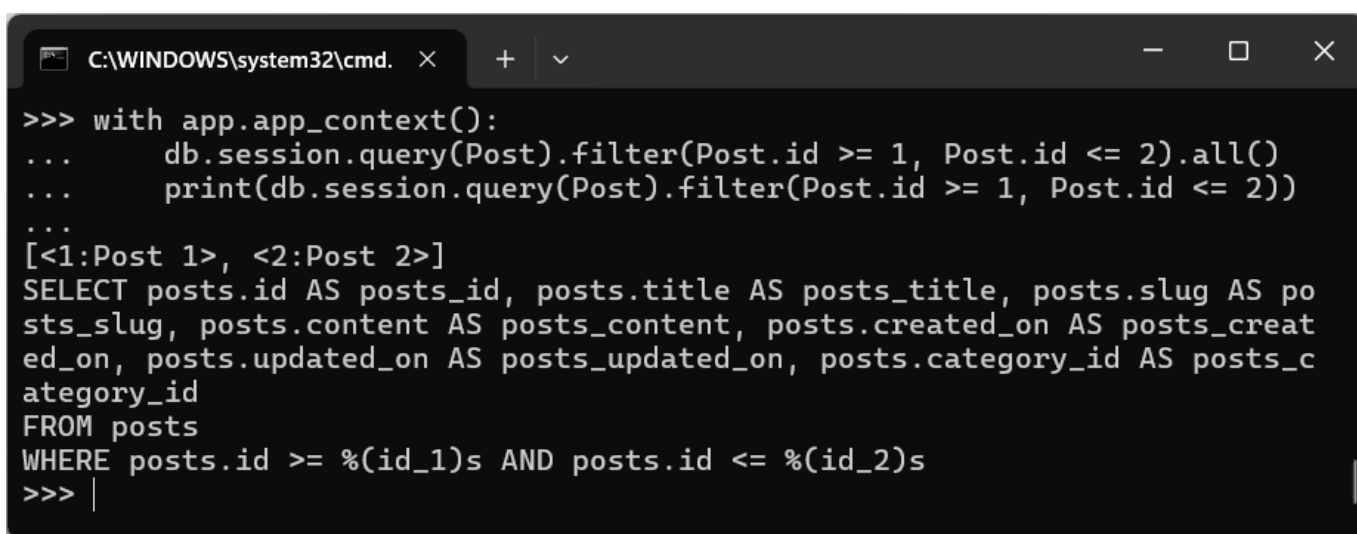
2.3.6 Метод `filter()`

Метод `filter()` дає змогу впорядкувати результати за допомогою оператора *WHERE*, застосованого до запиту. Він приймає колонку, оператор або значення. Наприклад:

```
C:\WINDOWS\system32\cmd. x + v
>>> with app.app_context():
...     db.session.query(Post).filter(Post.title == 'Post 1').all()
...     print(db.session.query(Post).filter(Post.title == 'Post 1'))
...
[<1:Post 1>]
SELECT posts.id AS posts_id, posts.title AS posts_title, posts.slug AS po
sts_slug, posts.content AS posts_content, posts.created_on AS posts_creat
ed_on, posts.updated_on AS posts_updated_on, posts.category_id AS posts_c
ategory_id
FROM posts
WHERE posts.title = %(title_1)s
>>> |
```


Запит поверне всі пости із заголовком *"Post 1"*. SQL-еквівалент запиту наступний. Рядок *% (title_1)* s в умові *WHERE* - це заповнювач. На його місці буде реальне значення під час виконання запиту.

Методу *filter()* можна передати кілька значень і вони будуть об'єднані оператором *AND* в SQL. Наприклад:



```
C:\WINDOWS\system32\cmd. x + v
>>> with app.app_context():
...     db.session.query(Post).filter(Post.id >= 1, Post.id <= 2).all()
...     print(db.session.query(Post).filter(Post.id >= 1, Post.id <= 2))
...
[<1:Post 1>, <2:Post 2>]
SELECT posts.id AS posts_id, posts.title AS posts_title, posts.slug AS po
sts_slug, posts.content AS posts_content, posts.created_on AS posts_creat
ed_on, posts.updated_on AS posts_updated_on, posts.category_id AS posts_c
ategory_id
FROM posts
WHERE posts.id >= %(id_1)s AND posts.id <= %(id_2)s
>>> |
```


2.3.7 Метод `first_or_404()`

Робить те ж саме, що і метод `first()`, але замість `None` повертає помилку `404`, якщо запит без результату.

```
C:\WINDOWS\system32\cmd. x + v - □ ×
>>> with app.app_context():
...     db.session.query(Post).filter(Post.id > 1).first_or_404()
...     db.session.query(Post).filter(Post.id > 10).first_or_404().all()
...
<2:Post 2>
Traceback (most recent call last):
  File "<stdin>", line 3, in <module>
  File "c:\Flask\.venv\Lib\site-packages\flask_sqlalchemy\query.py", line
  46, in first_or_404
    abort(404, description=description)
  File "c:\Flask\.venv\Lib\site-packages\flask\helpers.py", line 272, in
  abort
    current_app.aborter(code, *args, **kwargs)
  File "c:\Flask\.venv\Lib\site-packages\werkzeug\exceptions.py", line 86
  1, in __call__
    raise self.mapping[code](*args, **kwargs)
werkzeug.exceptions.NotFound: 404 Not Found: The requested URL was not fo
und on the server. If you entered the URL manually please check your spel
ling and try again.
>>> |
```

2.3.8 Метод `limit()`

Метод `limit()` додає оператор *LIMIT* до запиту. Він приймає кількість рядків, які потрібно повернути із запитом. SQL-еквівалент:

```
C:\WINDOWS\system32\cmd. x + v
>>> with app.app_context():
...     db.session.query(Post).limit(2).all()
...     print(db.session.query(Post).limit(2))
...     db.session.query(Post).filter(Post.id >= 2).limit(1).all()
...     print(db.session.query(Post).filter(Post.id >= 2).limit(1))
...
[<1:Post 1>, <2:Post 2>]
SELECT posts.id AS posts_id, posts.title AS posts_title, posts.slug AS po
sts_slug, posts.content AS posts_content, posts.created_on AS posts_creat
ed_on, posts.updated_on AS posts_updated_on, posts.category_id AS posts_c
ategory_id
FROM posts
LIMIT %(param_1)s
[<2:Post 2>]
SELECT posts.id AS posts_id, posts.title AS posts_title, posts.slug AS po
sts_slug, posts.content AS posts_content, posts.created_on AS posts_creat
ed_on, posts.updated_on AS posts_updated_on, posts.category_id AS posts_c
ategory_id
FROM posts
WHERE posts.id >= %(id_1)s
LIMIT %(param_1)s
>>> |
```

2.3.9 Метод `offset()`

Метод `offset()` додає умову *OFFSET* у запит. Як аргумент він приймає зміщення. Часто використовується разом із `limit()`.

```
C:\WINDOWS\system32\cmd. x + v
>>> with app.app_context():
...     db.session.query(Post).filter(Post.id > 1).limit(3).offset(1).all()
...
[<3:Post 3>, <4:Post 4>]
>>> |
```

SQL-еквівалент:

```
C:\WINDOWS\system32\cmd. x + v
>>> with app.app_context():
...     print(db.session.query(Post).filter(Post.id > 1).limit(3).offset(1))
...
SELECT posts.id AS posts_id, posts.title AS posts_title, posts.slug AS posts_
slug, posts.content AS posts_content, posts.created_on AS posts_created_on, p
osts.updated_on AS posts_updated_on, posts.category_id AS posts_category_id
FROM posts
WHERE posts.id > %(id_1)s
LIMIT %(param_1)s, %(param_2)s
>>> |
```

Рядки `%(param_1)s` і `%(param_2)s` - заповнювачі для зсуву та обмеження виводу, відповідно.

2.3.10 Метод `order_by()`

Метод `order_by()` використовується, щоб упорядкувати результат, додавши до запиту оператор *ORDER BY*. Він приймає кількість колонок, для яких потрібно встановити порядок. За замовчуванням сортує в порядку зростання.

```
C:\WINDOWS\system32\cmd. x + v
>>> with app.app_context():
...     db.session.query(Tag).all()
...     db.session.query(Tag).order_by(Tag.name).all()
...
[<built-in function id>:refactoring>, <built-in function id>:snippet>, <bu
ilt-in function id>:analytics>]
[<built-in function id>:analytics>, <built-in function id>:refactoring>, <
built-in function id>:snippet>]
>>> |
```

Для сортування за спаданням потрібно використовувати функцію `db.desc()`:

```

C:\WINDOWS\system32\cmd. x + v - □ x
>>> with app.app_context():
...     db.session.query(Tag).order_by(db.desc(Tag.name)).all()
...
[<<built-in function id>:snippet>, <<built-in function id>:refactoring>, <<bu
ilt-in function id>:analytics>]
>>> |

```

2.3.11 Метод join()

Метод *join()* використовується для створення *JOIN* у SQL. Він приймає ім'я таблиці, для якої потрібно створити *JOIN*.

```

C:\WINDOWS\system32\cmd. x + v - □ x
>>> with app.app_context():
...     db.session.query(Post).join(Category).all()
...     print(db.session.query(Post).join(Category))
...
[<1:Post 1>, <2:Post 2>, <3:Post 3>]
SELECT posts.id AS posts_id, posts.title AS posts_title, posts.slug AS posts_
slug, posts.content AS posts_content, posts.created_on AS posts_created_on, p
osts.updated_on AS posts_updated_on, posts.category_id AS posts_category_id
FROM posts INNER JOIN categories ON categories.id = posts.category_id
>>> |

```

Метод *join()* широко використовується, щоб отримати дані з однієї або більшої кількості таблиць одним запитом. Наприклад:

```

C:\WINDOWS\system32\cmd. x + v - □ x
>>> with app.app_context():
...     db.session.query(Post.title, Category.name).join(Category).all()
...
[('Post 1', 'Python'), ('Post 2', 'Python'), ('Post 3', 'Java')]
>>> |

```

2.4 Створення контактної форми

Закінчити тему можна завершенням контактної форми.

Варто нагадати, що в темі "Робота з формами у Flask" було створено контактну форму для отримання зворотного зв'язку від користувачів. Поки що функція подання *contact()* не зберігає надіслані дані. Вона тільки виводить їх у консолі. Для збереження отриманої інформації спочатку потрібно створити нову таблицю. Відкриємо *main.py*, щоб додати модель *Feedback* слідом за моделлю *Tag*:

```
class Feedback(db.Model):
    __tablename__ = 'feedbacks'
    id = db.Column(db.Integer(), primary_key=True)
    name = db.Column(db.String(1000), nullable=False)
    email = db.Column(db.String(100), nullable=False)
    message = db.Column(db.Text(), nullable=False)
    created_on = db.Column(db.DateTime(), default=datetime.utcnow)

    def __repr__(self):
        return "<{}:{}>".format(self.id, self.name)
```

Далі потрібно перезапустити оболонку Python і викликати метод *create_all()* об'єкта *db* для створення таблиці *feedbacks*:

```
from main import app, db
with app.app_context():
    db.create_all()
```

Також потрібно змінити функцію подання *contact()*:

```
@app.route('/contact/', methods=['get', 'post'])
def contact():
    form = ContactForm()
    if form.validate_on_submit():
        name = form.name.data
        email = form.email.data
        message = form.message.data
        print(name)
        print(Post)
        print(email)
        print(message)

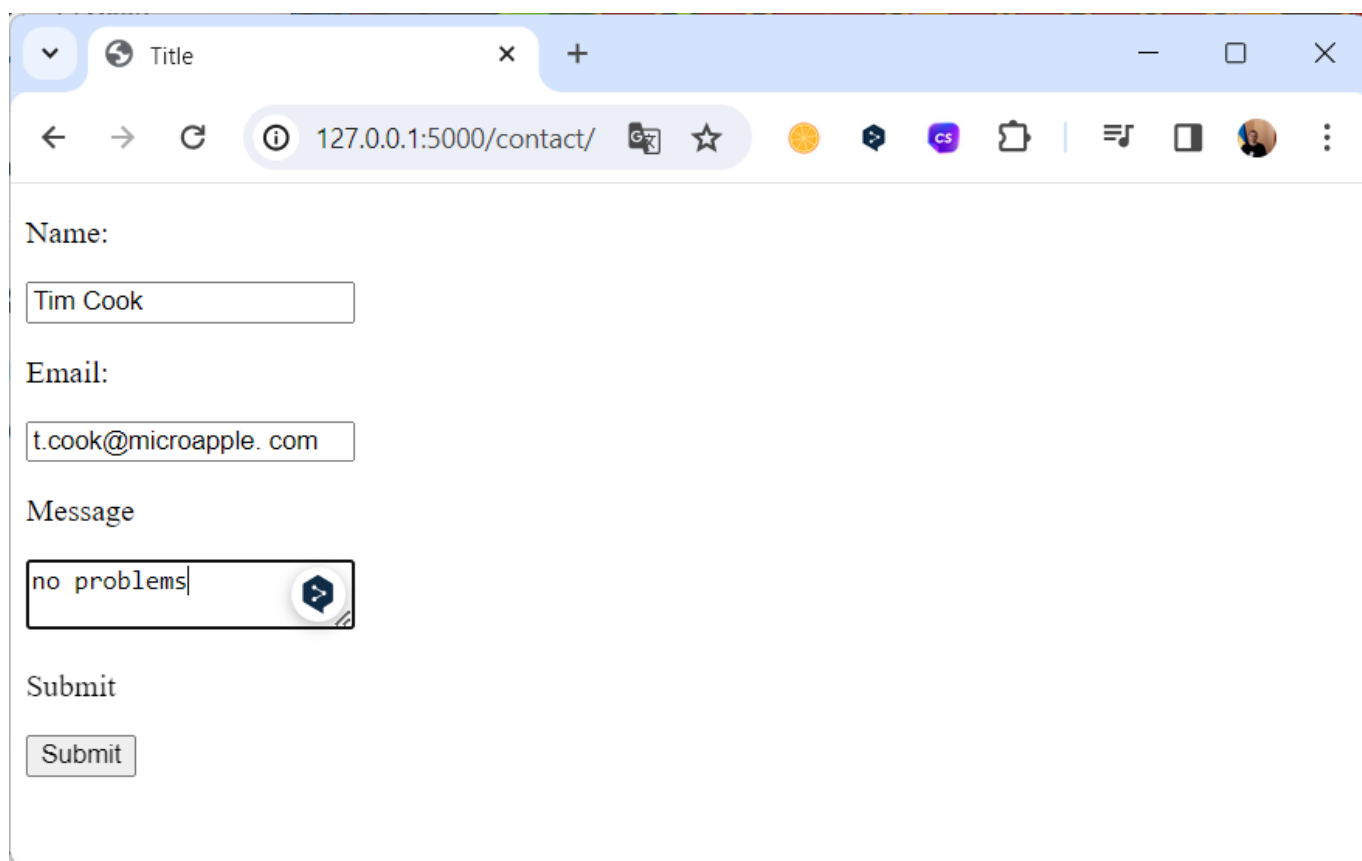
    # реалізація логіки бази даних
```

```
feedback = Feedback(name=name, email=email, message=message)
db.session.add(feedback)
db.session.commit()

print("\nData received. Now redirecting ...")
flash("Message Received", "success")
return redirect(url_for('contact'))

return render_template('contact.html', form=form)
```

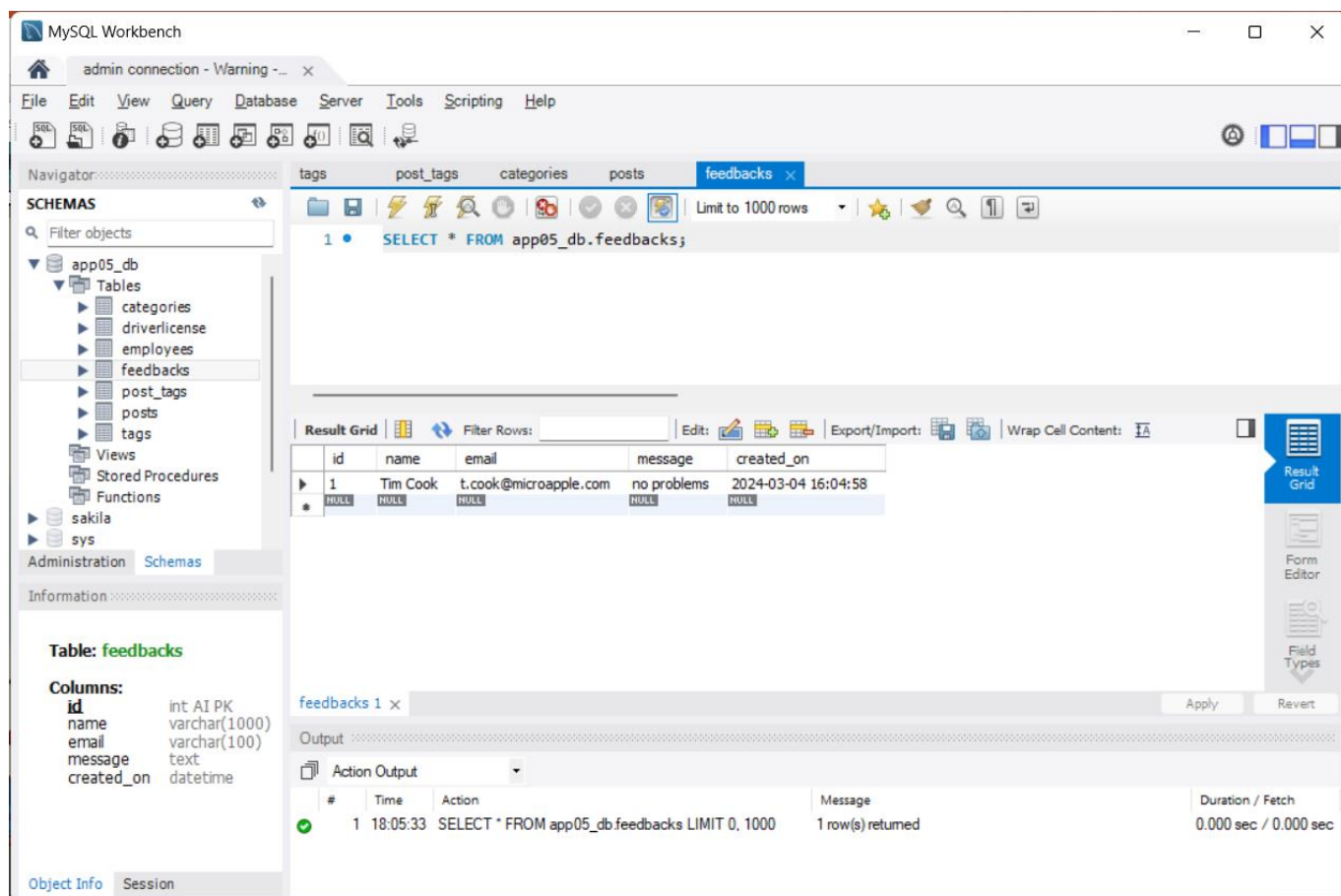
Запустимо сервер і заїдемо на <http://127.0.0.1:5000/contact/>, щоб заповнити та надіслати форму.



The screenshot shows a web browser window with the address bar displaying `127.0.0.1:5000/contact/`. The page contains a contact form with the following elements:

- Name:** A text input field containing "Tim Cook".
- Email:** A text input field containing "t.cook@microapple.com".
- Message:** A text area containing "no problems" with a small icon on the right.
- Submit:** A button labeled "Submit" at the bottom of the form.

Відправлений запис матиме такий наступний вигляд:



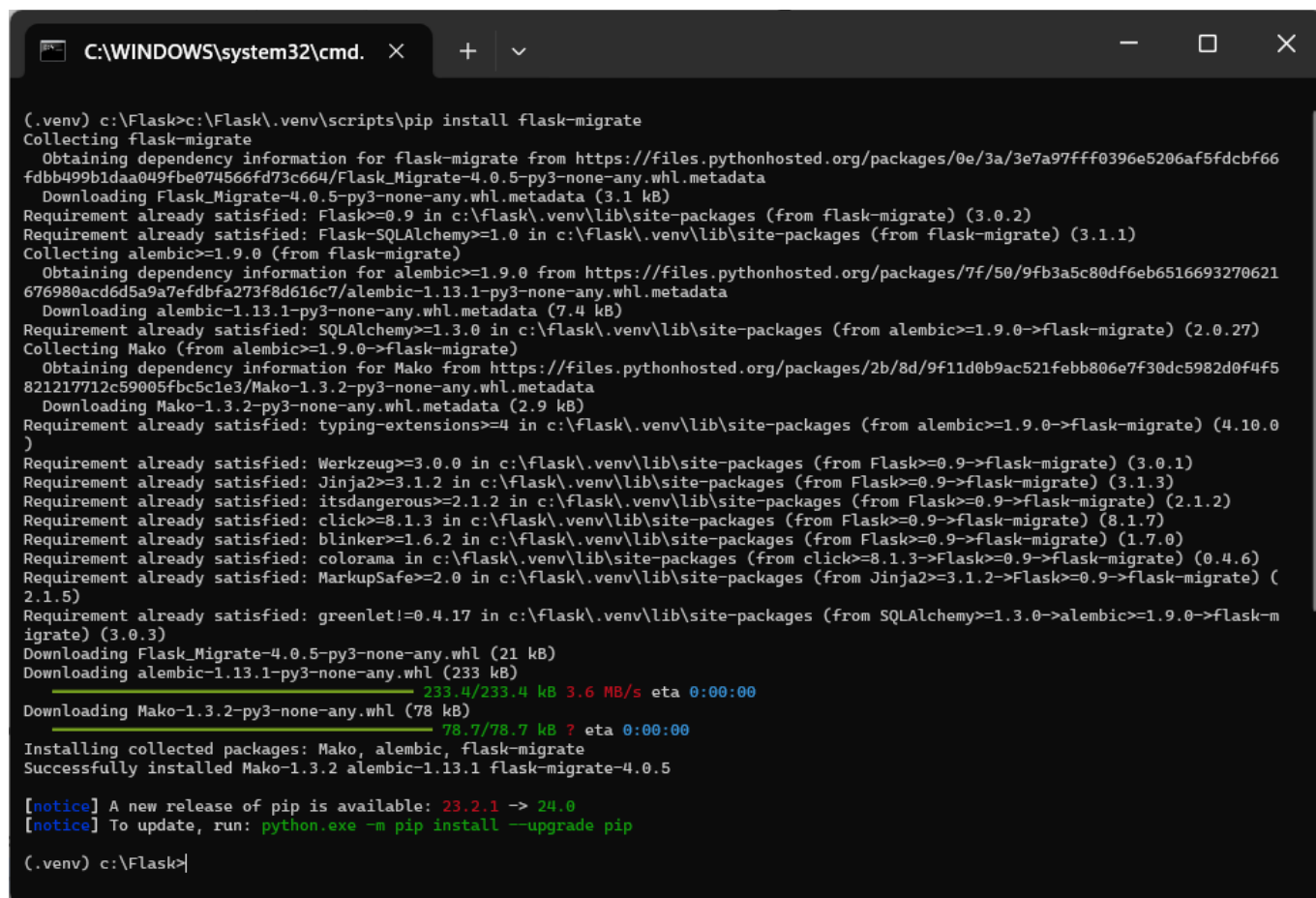
З МІГРАЦІЇ БАЗИ ДАНИХ ЗА ДОПОМОГОЮ ALEMBIC

Alembic - це інструмент для міграції бази даних, який використовується в SQLAlchemy. Міграція бази даних - це щось схоже на систему контролю версій для баз даних. Варто нагадати, що метод *create_all()* у SQLAlchemy лише створює відсутні таблиці з моделей. Коли таблиця вже створена, він не змінює її схему, ґрунтуючись на змінах у моделі.

Під час розроблення застосунку поширена практика зміни схеми таблиці. Тут і приходить на допомогу Alembic. Він, як і інші подібні інструменти, дає змогу змінювати схему бази даних під час розвитку програми. Він також стежить за змінами самої бази, тож можна рухатися туди й назад. Якщо не використовувати Alembic, то за всіма змінами доведеться стежити вручну і змінювати схему за допомогою *Alter*.

3.1 Розширення Flask-Migrate

Flask-Migrate - це розширення, яке інтегрує Alembic у додаток Flask. Встановити його можна за допомогою такої команди.



```
(.venv) c:\Flask>c:\Flask\.venv\scripts\pip install flask-migrate
Collecting flask-migrate
  Obtaining dependency information for flask-migrate from https://files.pythonhosted.org/packages/0e/3a/3e7a97fff0396e5206af5fdbc6f66fdbb499b1daa049f9be074566fd73c664/Flask_Migrate-4.0.5-py3-none-any.whl.metadata
  Downloading Flask_Migrate-4.0.5-py3-none-any.whl.metadata (3.1 kB)
Requirement already satisfied: Flask>=0.9 in c:\flask\.venv\lib\site-packages (from flask-migrate) (3.0.2)
Requirement already satisfied: Flask-SQLAlchemy>=1.0 in c:\flask\.venv\lib\site-packages (from flask-migrate) (3.1.1)
Collecting alembic>=1.9.0 (from flask-migrate)
  Obtaining dependency information for alembic>=1.9.0 from https://files.pythonhosted.org/packages/7f/50/9fb3a5c80df6eb6516693270621676980acd6d5a9a7efdbfa273f8d616c7/alembic-1.13.1-py3-none-any.whl.metadata
  Downloading alembic-1.13.1-py3-none-any.whl.metadata (7.4 kB)
Requirement already satisfied: SQLAlchemy>=1.3.0 in c:\flask\.venv\lib\site-packages (from alembic>=1.9.0->flask-migrate) (2.0.27)
Collecting Mako (from alembic>=1.9.0->flask-migrate)
  Obtaining dependency information for Mako from https://files.pythonhosted.org/packages/2b/8d/9f11d0b9ac521febb806e7f30dc5982d0f4f5821217712c59005fbc5c1e3/Mako-1.3.2-py3-none-any.whl.metadata
  Downloading Mako-1.3.2-py3-none-any.whl.metadata (2.9 kB)
Requirement already satisfied: typing-extensions>=4 in c:\flask\.venv\lib\site-packages (from alembic>=1.9.0->flask-migrate) (4.10.0)
Requirement already satisfied: Werkzeug>=3.0.0 in c:\flask\.venv\lib\site-packages (from Flask>=0.9->flask-migrate) (3.0.1)
Requirement already satisfied: Jinja2>=3.1.2 in c:\flask\.venv\lib\site-packages (from Flask>=0.9->flask-migrate) (3.1.3)
Requirement already satisfied: itsdangerous>=2.1.2 in c:\flask\.venv\lib\site-packages (from Flask>=0.9->flask-migrate) (2.1.2)
Requirement already satisfied: click>=8.1.3 in c:\flask\.venv\lib\site-packages (from Flask>=0.9->flask-migrate) (8.1.7)
Requirement already satisfied: blinker>=1.6.2 in c:\flask\.venv\lib\site-packages (from Flask>=0.9->flask-migrate) (1.7.0)
Requirement already satisfied: colorama in c:\flask\.venv\lib\site-packages (from click>=8.1.3->Flask>=0.9->flask-migrate) (0.4.6)
Requirement already satisfied: MarkupSafe>=2.0 in c:\flask\.venv\lib\site-packages (from Jinja2>=3.1.2->Flask>=0.9->flask-migrate) (2.1.5)
Requirement already satisfied: greenlet!=0.4.17 in c:\flask\.venv\lib\site-packages (from SQLAlchemy>=1.3.0->alembic>=1.9.0->flask-migrate) (3.0.3)
Downloading Flask_Migrate-4.0.5-py3-none-any.whl (21 kB)
Downloading alembic-1.13.1-py3-none-any.whl (233 kB)
233.4/233.4 kB 3.6 MB/s eta 0:00:00
Downloading Mako-1.3.2-py3-none-any.whl (78 kB)
78.7/78.7 kB ? eta 0:00:00
Installing collected packages: Mako, alembic, flask-migrate
Successfully installed Mako-1.3.2 alembic-1.13.1 flask-migrate-4.0.5

[notice] A new release of pip is available: 23.2.1 -> 24.0
[notice] To update, run: python.exe -m pip install --upgrade pip

(.venv) c:\Flask>
```

Для інтеграції Flask-Migrate із застосунком потрібно імпортувати клас *Migrate* з пакета *flask_package*, а також створити екземпляр класу *Migrate*, передавши екземпляр застосунку (*app*) і об'єкт SQLAlchemy (*db*):


```

from flask_migrate import Migrate

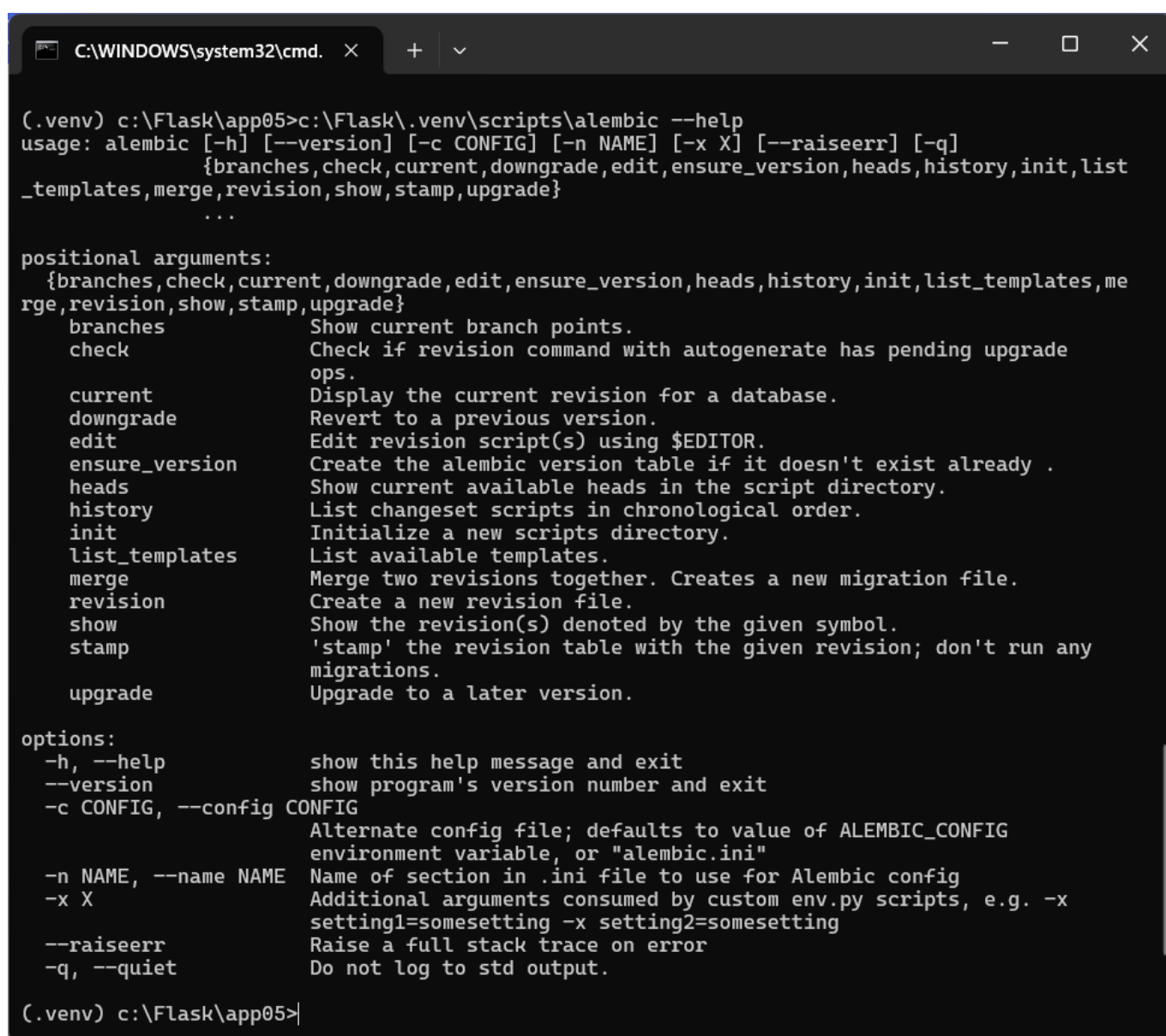
app = Flask(__name__)

app.config.update(
    DEBUG=True,
    SECRET_KEY='strong secret key',
    SQLALCHEMY_DATABASE_URI = 'mysql+pymysql://admin:masterkey@localhost/app05_db'
)

db = SQLAlchemy(app)
migrate = Migrate(app, db)

```

Щоб подивитися повний список команд для Alembic, потрібно ввести таке:



```

C:\WINDOWS\system32\cmd. x + v

(.venv) c:\Flask\app05>c:\Flask\.venv\scripts\alembic --help
usage: alembic [-h] [--version] [-c CONFIG] [-n NAME] [-x X] [--raiseerr] [-q]
               {branches,check,current,downgrade,edit,ensure_version,heads,history,init,list_
_templates,merge,revision,show,stamp,upgrade}
               ...

positional arguments:
  {branches,check,current,downgrade,edit,ensure_version,heads,history,init,list_templates,me
rge,revision,show,stamp,upgrade}
    branches           Show current branch points.
    check              Check if revision command with autogenerate has pending upgrade
ops.
    current            Display the current revision for a database.
    downgrade          Revert to a previous version.
    edit               Edit revision script(s) using $EDITOR.
    ensure_version     Create the alembic version table if it doesn't exist already .
    heads              Show current available heads in the script directory.
    history             List changeset scripts in chronological order.
    init               Initialize a new scripts directory.
    list_templates     List available templates.
    merge              Merge two revisions together. Creates a new migration file.
    revision           Create a new revision file.
    show               Show the revision(s) denoted by the given symbol.
    stamp              'stamp' the revision table with the given revision; don't run any
migrations.
    upgrade            Upgrade to a later version.

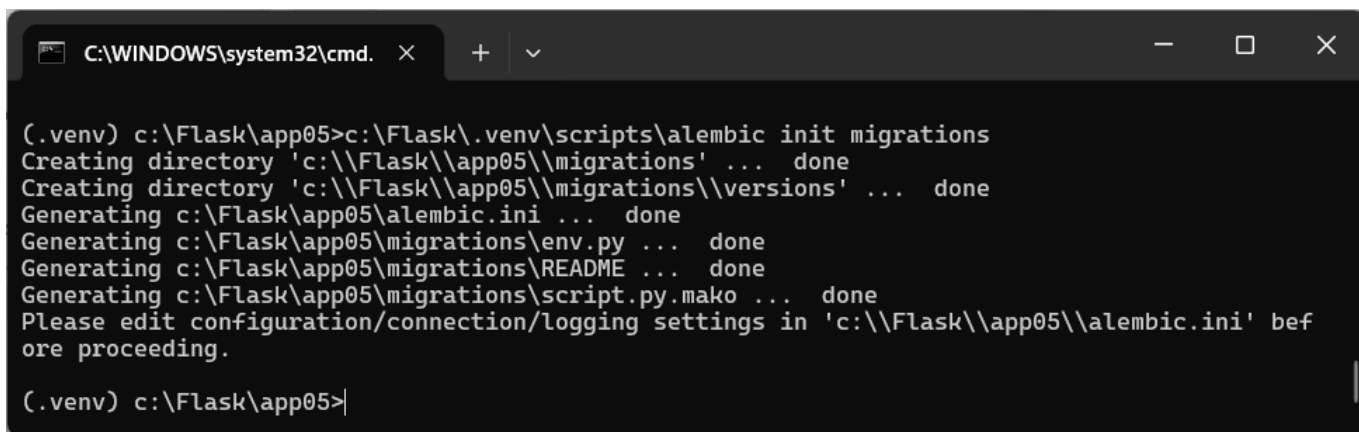
options:
  -h, --help            show this help message and exit
  --version             show program's version number and exit
  -c CONFIG, --config CONFIG
                       Alternate config file; defaults to value of ALEMBIC_CONFIG
environment variable, or "alembic.ini"
  -n NAME, --name NAME  Name of section in .ini file to use for Alembic config
  -x X                  Additional arguments consumed by custom env.py scripts, e.g. -x
setting1=somesetting -x setting2=somesetting
  --raiseerr            Raise a full stack trace on error
  -q, --quiet           Do not log to std output.

(.venv) c:\Flask\app05>

```

Це реальні команди, які будуть використовуватися для міграцій бази даних.

Перед тим як Alembic почне відстежувати зміни, потрібно встановити репозиторій міграції. Репозиторій міграції - це всього лише папка, яка містить налаштування Alembic і скрипти міграції. Для створення репозиторію потрібно виконати команду *init*:



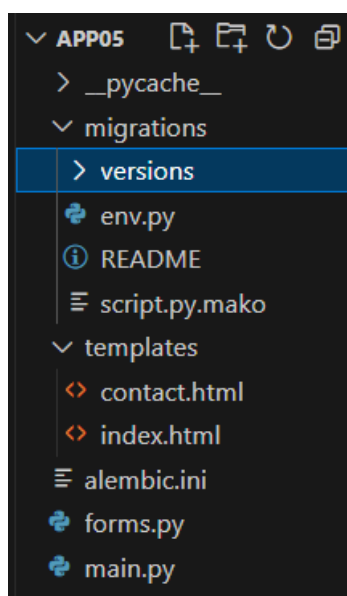
```

C:\WINDOWS\system32\cmd. x + v
(.venv) c:\Flask\app05>c:\Flask\.venv\scripts\alembic init migrations
Creating directory 'c:\\Flask\\app05\\migrations' ... done
Creating directory 'c:\\Flask\\app05\\migrations\\versions' ... done
Generating c:\Flask\app05\alembic.ini ... done
Generating c:\Flask\app05\migrations\env.py ... done
Generating c:\Flask\app05\migrations\README ... done
Generating c:\Flask\app05\migrations\script.py.mako ... done
Please edit configuration/connection/logging settings in 'c:\\Flask\\app05\\alembic.ini' before proceeding.

(.venv) c:\Flask\app05>

```

Ця команда створить папку "*migrations*" всередині папки додатка *app05*. Структура папки *migrations* наступна:



Короткий опис кожної папки та файлу:

- *alembic.ini* - файл із налаштування Alembic.
- *env.py* - файл Python, який запускається щоразу, коли викликається Alembic.

Він з'єднується з базою даних, запускає транзакцію і викликає рушій міграції.

- *README* - файл README.
- *script.py.mako* - файл шаблону Мако, який буде використовуватися для створення скриптів міграції.
- *version* - папка для зберігання скриптів міграції.

3.2 Створення скрипта міграції

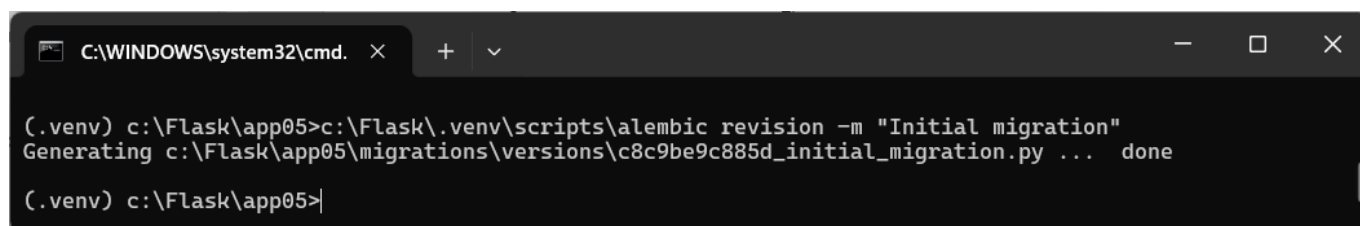
Alembic зберігає міграції бази даних у скриптах міграції, які являють собою звичайні файли Python. Скрипт міграції визначає дві функції: *upgrade()* і *downgrade()*. Завдання *upgrade()* - застосувати зміни до бази даних, а *downgrade()* - відкотити їх назад. Коли міграція застосовується, викликається функція *upgrade()*. При поверненні назад - *downgrade()*.

Alembic пропонує два варіанти створення міграцій:

- Вручну за допомогою команди *revision*.
- Автоматично за допомогою команди *migrate*.

Ручна або порожня міграція створює скрипт міграції з порожніми функціями *upgrade()* і *downgrade()*. Завдання - заповнити їх за допомогою інструкцій Alembic, які і буде застосовувати зміни до бази даних. Ручна міграція використовується тоді, коли потрібен повний контроль над процесом міграції. Для створення порожньої міграції потрібно ввести таку команду:

```
alembic revision -m "Initial migration"
```



```
C:\WINDOWS\system32\cmd. x + v
(.venv) c:\Flask\app05>c:\Flask\.venv\scripts\alembic revision -m "Initial migration"
Generating c:\Flask\app05\migrations\versions\c8c9be9c885d_initial_migration.py ... done
(.venv) c:\Flask\app05>
```

Ця команда створить новий скрипт міграції в папці *migrations/version*. Назва файлу має бути у форматі *someid_initial_migration.py*. Файл має виглядати ось так:

```

1  """Initial migration
2
3  Revision ID: c8c9be9c885d
4  Revises:
5  Create Date: 2024-03-04 19:37:11.546341
6
7  """
8  from typing import Sequence, Union
9
10 from alembic import op
11 import sqlalchemy as sa
12
13 # revision identifiers, used by Alembic.
14 revision: str = 'c8c9be9c885d'
15 down_revision: Union[str, None] = None
16 branch_labels: Union[str, Sequence[str], None] = None
17 depends_on: Union[str, Sequence[str], None] = None
18
19
20 def upgrade() -> None:
21     pass
22
23
24 def downgrade() -> None:
25     pass
26

```

Він починається із закоментованої частини, яка містить повідомлення, задане за допомогою мітки *-m*, ID зміни та часу, коли файл було створено. Наступна важлива частина - ідентифікатори зміни. Кожен скрипт міграції отримує унікальний ID зміни, який зберігається у змінній *revision*. У наступному рядку є змінна *down_revision* зі значенням *None*. Alembic використовує змінну *down_revision*, щоб визначити, яку міграцію запускати і в якому порядку. Змінна *down_revision* вказує на ідентифікатор зміни батьківської міграції. У цьому випадку її значення - *None*, тому що це тільки перший скрипт міграції. Наприкінці файлу є порожні функції *upgrade()* і *downgrade()*.

Тепер потрібно відредагувати файл міграції, щоб додати операції створення і видалення таблиці для функцій *upgrade()* і *downgrade()*, відповідно.

У функції *upgrade()* використовується інструкція *create_table()* Alembic.

Інструкція *create_table()* використовує оператор *CREATE TABLE*.

Барбарук Віктор Миколайович. Сучасні технології розробки WEB-застосунків з використанням мови Python

У функції *downgrade()* інструкція *drop_table()* задіює оператор *DROP TABLE*.

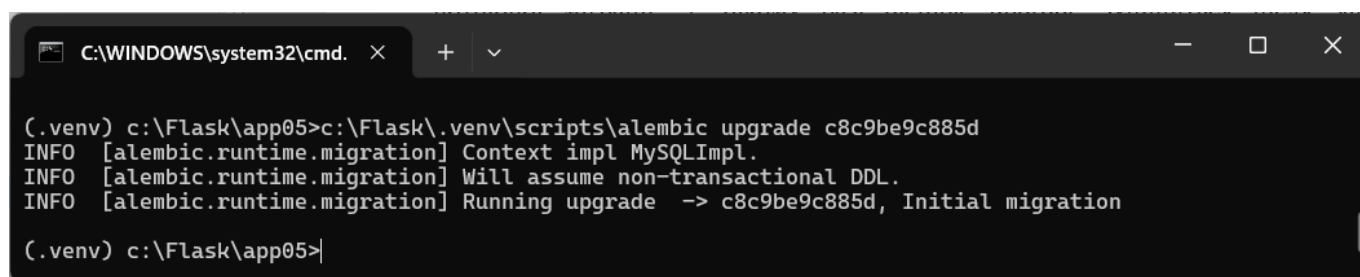
Під час першого запуску міграції буде створено таблицю *users*, а під час відкату - ця ж міграція видалить таблицю *users*.

Тепер можна виконати першу міграцію. Для цього потрібно ввести таку команду:

```
alembic upgrade
```

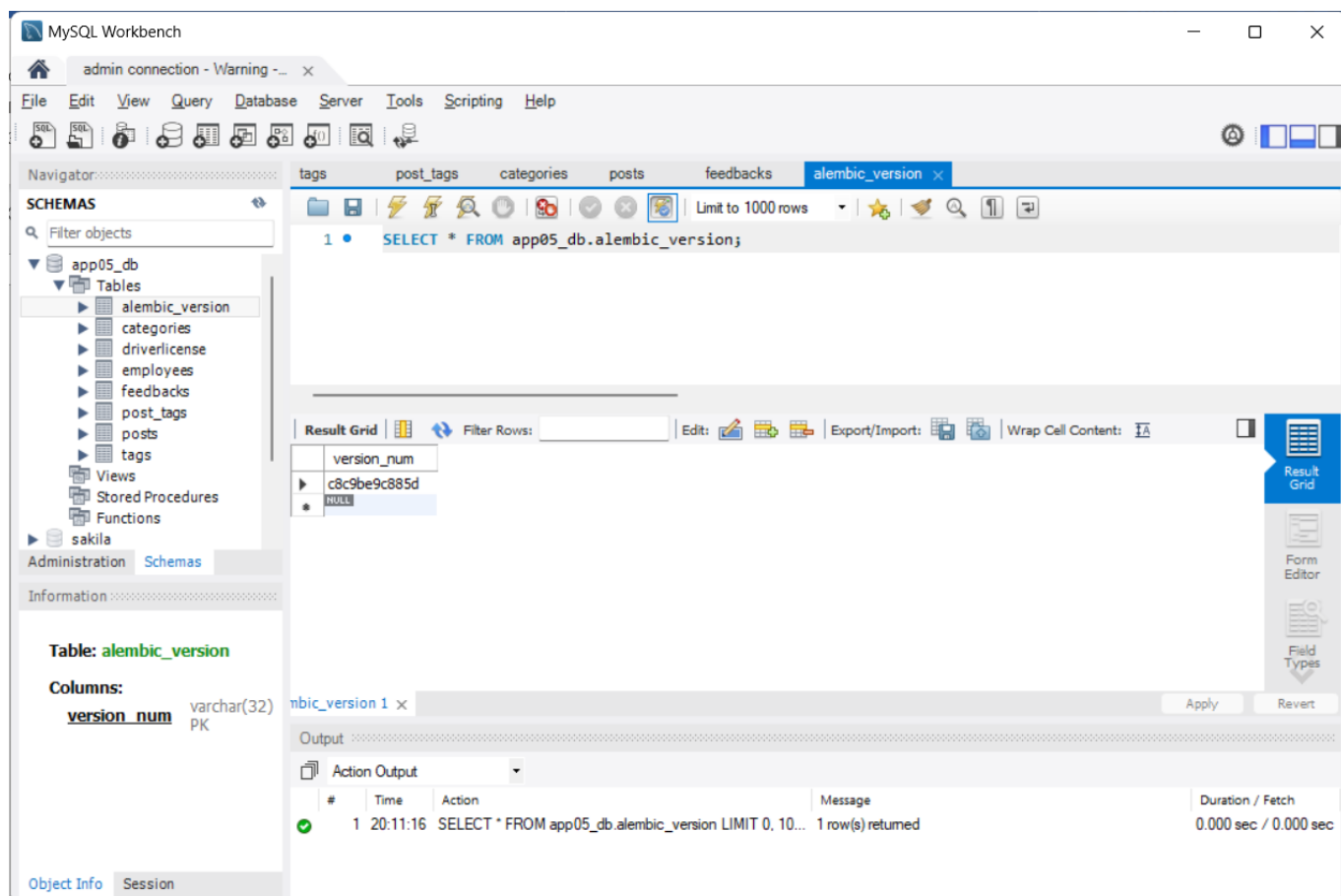
Ця команда виконає функцію *upgrade()* скрипта міграції. Команда *alembic upgrade* поверне базу даних до останньої міграції. Варто зауважити, що *alembic upgrade head* не тільки запускає останню міграцію, але й усі, які ще не були запущені. Це означає, що якщо міграцій було створено кілька, то *alembic upgrade head* запустить їх усі разом у порядку створення.

Замість запуску останньої міграції можна також передати ідентифікатор зміни потрібної міграції. У такому разі *alembic upgrade* зупиниться після запуску конкретної міграції і не буде виконувати наступні.

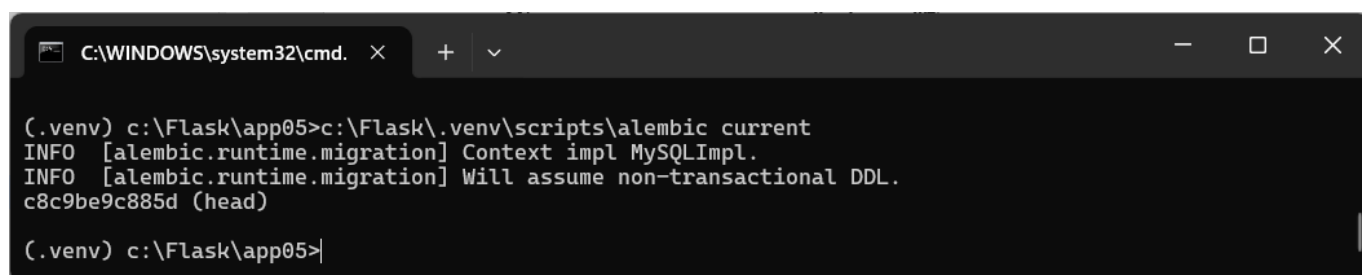


```
C:\WINDOWS\system32\cmd. x + v
(.venv) c:\Flask\app05>c:\Flask\.venv\scripts\alembic upgrade c8c9be9c885d
INFO [alembic.runtime.migration] Context impl MySQLImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.migration] Running upgrade -> c8c9be9c885d, Initial migration
(.venv) c:\Flask\app05>
```

Оскільки міграція запускається вперше, Alembic також створить таблицю *alembic_version*. Вона складається з однієї колонки *version_num*, яка зберігає ідентифікатор зміни останньої запущеної міграції. Саме так Alembic знає поточний стан міграції, і звідки її потрібно виконувати. Зараз таблиця *alembic_version* має ось такий вигляд:

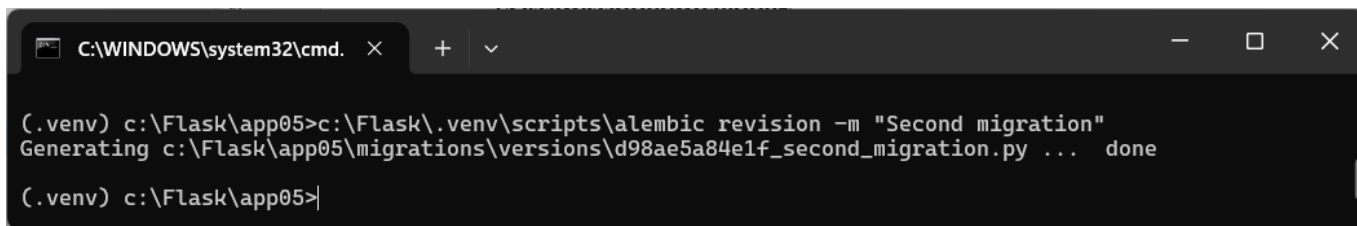


Визначити останню застосовану міграцію можна за допомогою команди *alembic current*. Вона поверне ідентифікатор зміни останньої міграції. Якщо такої не було, то нічого не повернеться.



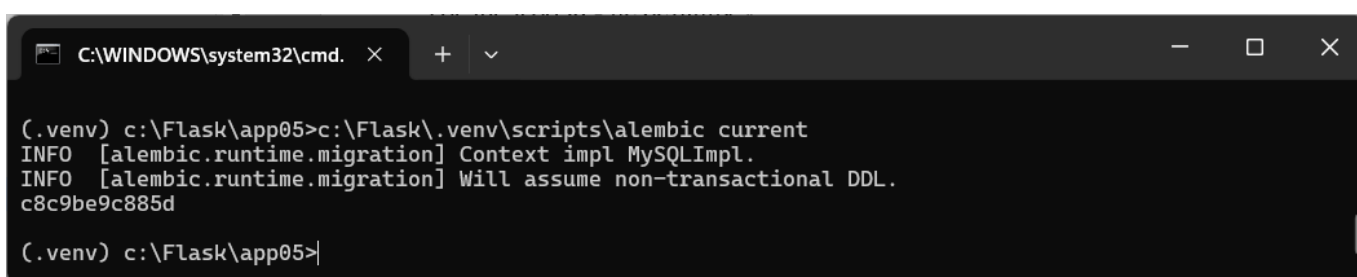
Виведення показує, що поточна міграція - *c8c9be9c885d*. Також потрібно звернути увагу на рядок (*head*) після ідентифікатора зміни, який вказує на те, що *c8c9be9c885d* - остання міграція.

Створимо ще одну порожню міграцію за допомогою команди *alembic revision*:



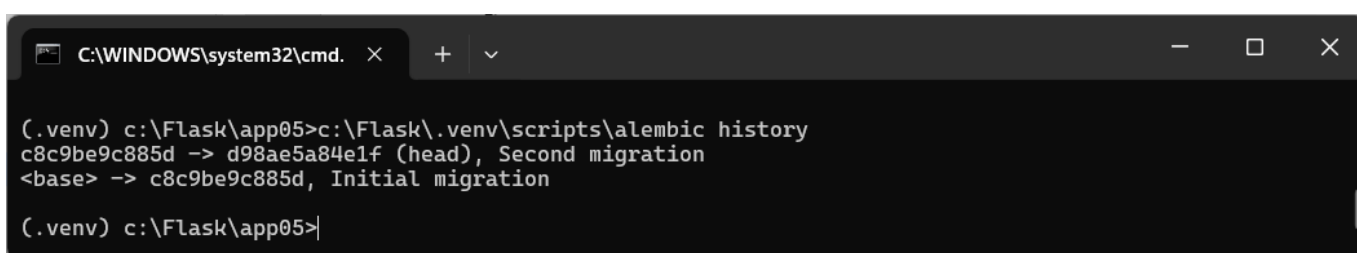
```
C:\WINDOWS\system32\cmd. x + v
(.venv) c:\Flask\app05>c:\Flask\.venv\scripts\alembic revision -m "Second migration"
Generating c:\Flask\app05\migrations\versions\d98ae5a84e1f_second_migration.py ... done
(.venv) c:\Flask\app05>
```

Далі потрібно знову запустити команду *alembic current*. Цього разу ідентифікатор зміни відображатиметься без рядка (*head*), тому що міграція *c8c9be9c885d* - не остання.



```
C:\WINDOWS\system32\cmd. x + v
(.venv) c:\Flask\app05>c:\Flask\.venv\scripts\alembic current
INFO [alembic.runtime.migration] Context impl MySQLImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
c8c9be9c885d
(.venv) c:\Flask\app05>
```

Щоб подивитися повний список міграцій (запущених і ні), потрібно використовувати команду *alembic history*. Вона поверне список міграцій у зворотному хронологічному порядку (остання міграція відображатиметься першою).



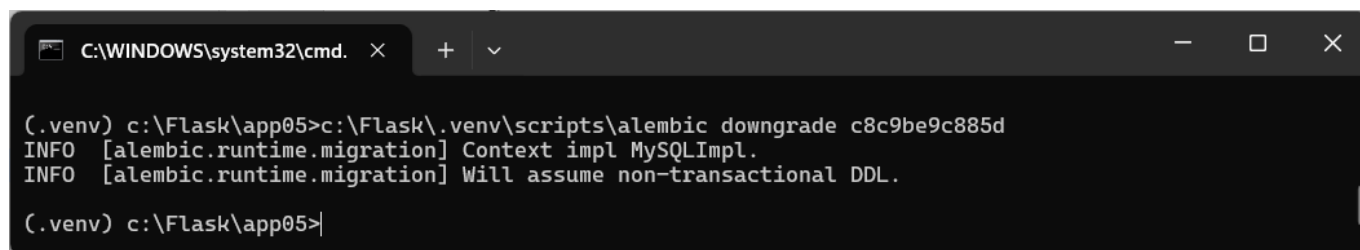
```
C:\WINDOWS\system32\cmd. x + v
(.venv) c:\Flask\app05>c:\Flask\.venv\scripts\alembic history
c8c9be9c885d -> d98ae5a84e1f (head), Second migration
<base> -> c8c9be9c885d, Initial migration
(.venv) c:\Flask\app05>
```

Висновок показує, що *c8c9be9c885d* - перша міграція, а слідом за нею йде *d98ae5a84e1f* - остання міграція. Як і зазвичай, (*head*) вказує на останню міграцію.

Повернути базу даних до початкового стану, який був до виконання команди *alembic upgrade*, можна за допомогою відкату міграції. Щоб відкотитися до останньої міграції, використовується команда *alembic downgrade*.

alembic downgrade

Як і у випадку з командою *alembic upgrade*, можна передати ідентифікатор зміни міграції, до якої потрібно відкотитися. Наприклад, щоб відкотитися до міграції *c8c9be9c885d*, потрібно використовувати таку команду:

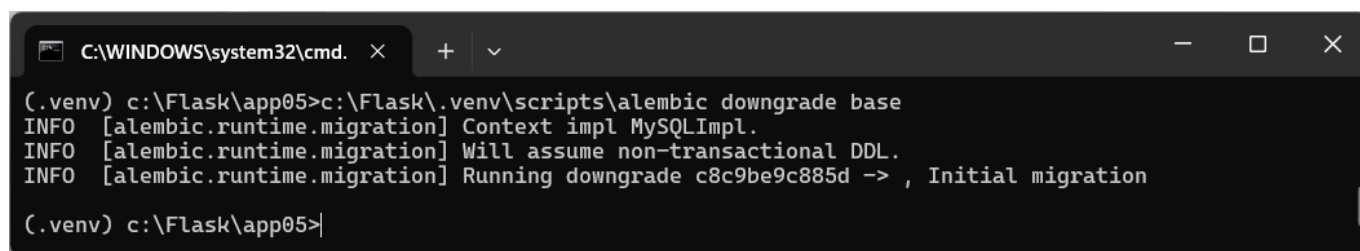


```

C:\WINDOWS\system32\cmd. x + v
(.venv) c:\Flask\app05>c:\Flask\.venv\scripts\alembic downgrade c8c9be9c885d
INFO [alembic.runtime.migration] Context impl MySQLImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
(.venv) c:\Flask\app05>

```

Щоб повернути всі прийняті міграції, потрібно використовувати таку команду:

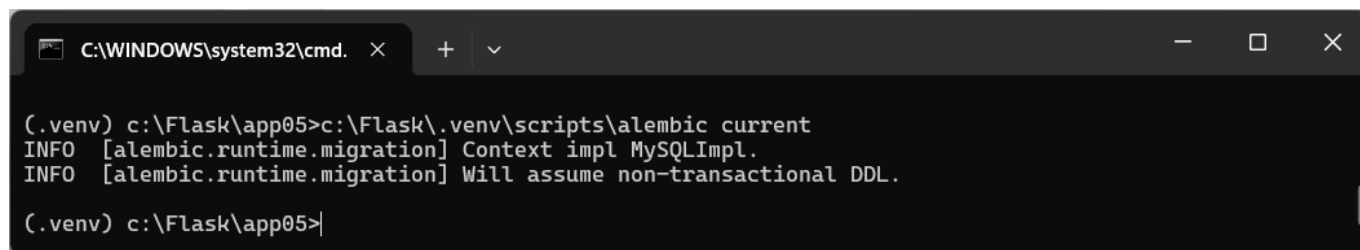


```

C:\WINDOWS\system32\cmd. x + v
(.venv) c:\Flask\app05>c:\Flask\.venv\scripts\alembic downgrade base
INFO [alembic.runtime.migration] Context impl MySQLImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.migration] Running downgrade c8c9be9c885d -> , Initial migration
(.venv) c:\Flask\app05>

```

Зараз до бази даних не застосовано жодної міграції. Переконатися в цьому можна, запустивши команду *alembic current*:

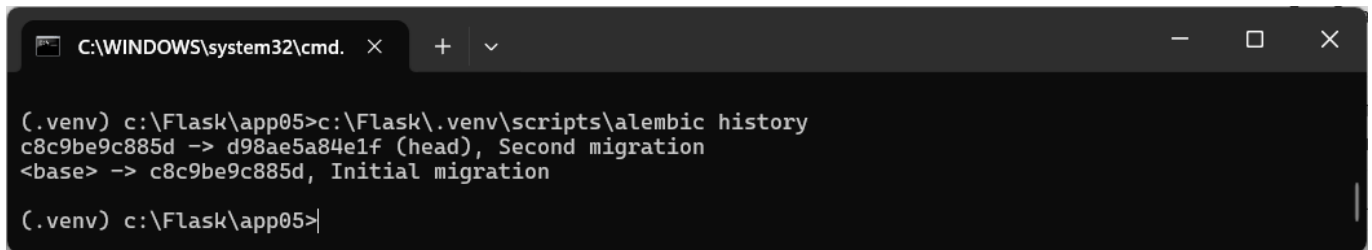


```

C:\WINDOWS\system32\cmd. x + v
(.venv) c:\Flask\app05>c:\Flask\.venv\scripts\alembic current
INFO [alembic.runtime.migration] Context impl MySQLImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
(.venv) c:\Flask\app05>

```

Як видно, виведення не повертає ідентифікатор зміни. Варто звернути увагу, що відкат міграції лише скасовує зміни бази даних, але не видаляє сам скрипт міграції. У результаті команда *alembic history* покаже два скрипти міграції.



```
C:\WINDOWS\system32\cmd. x + v
(.venv) c:\Flask\app05>c:\Flask\.venv\scripts\alembic history
c8c9be9c885d -> d98ae5a84e1f (head), Second migration
<base> -> c8c9be9c885d, Initial migration
(.venv) c:\Flask\app05>
```

Що буде, якщо зараз запустити команду *alembic upgrade*?

Команда *alembic upgrade* насамперед запустить міграцію *c8c9be9c885d*, а слідом за нею - *d98ae5a84e1f*.

База даних знову в початковому стані, а оскільки зміни в скриптах міграції не потрібні, їх можна видалити.